R3.04 – Qualité de développement

JavaFX

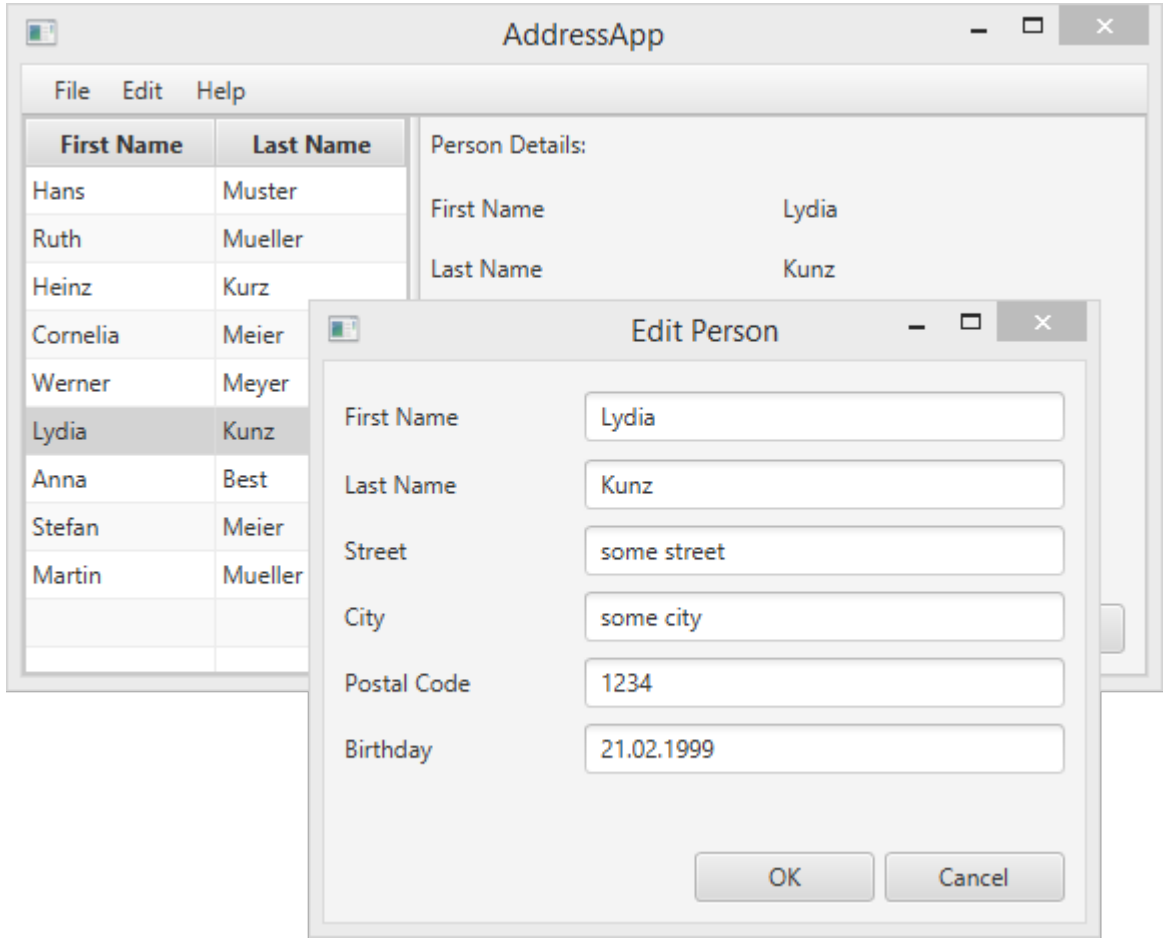
Sébastien Thon

BUT Informatique 2ème année
IUT d'Aix-Marseille Université, site d’Arles
Département Informatique

Introduction

Nous avons jusque là créé des applications en mode texte, fonctionnant dans un terminal.

Nous allons maintenant créer des applications présentant une interface graphique (**GUI** : *Graphical User Interface*), dans une ou plusieurs fenêtres.



Introduction

Plusieurs bibliothèques graphiques sont utilisables avec Java :

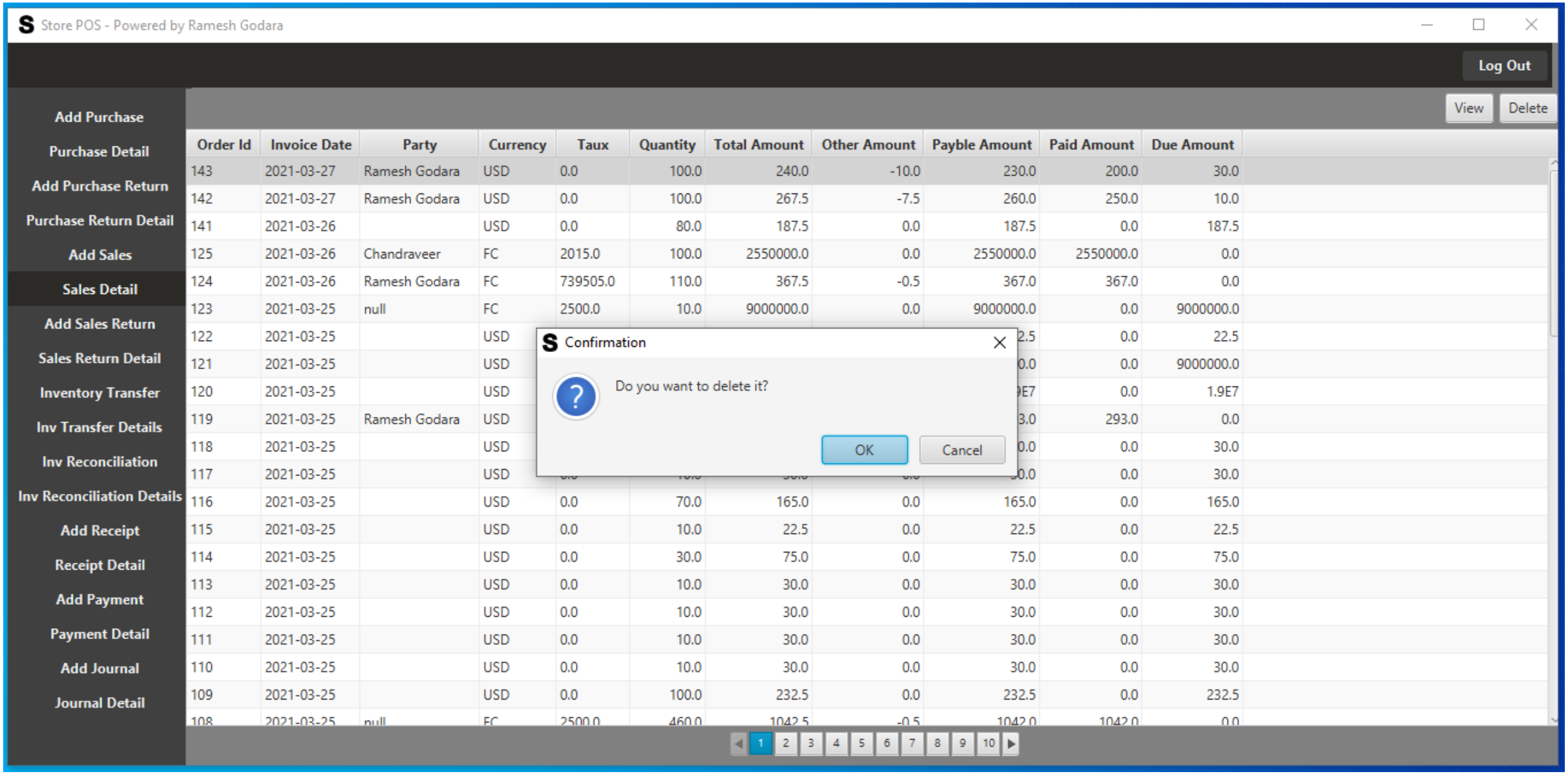
- AWT (Abstract Window Toolkit), historiquement la plus ancienne, intégrée dans le JDK de base.
- SWING, basée sur AWT, intégrée dans le JDK de base.
- SWT (Standard Widget Toolkit), open source https://eclipse.dev/eclipse/swt/
- JavaFX

Nous allons dans ce chapitre voir **JavaFX**, qui a pour avantage d'être la plus moderne.

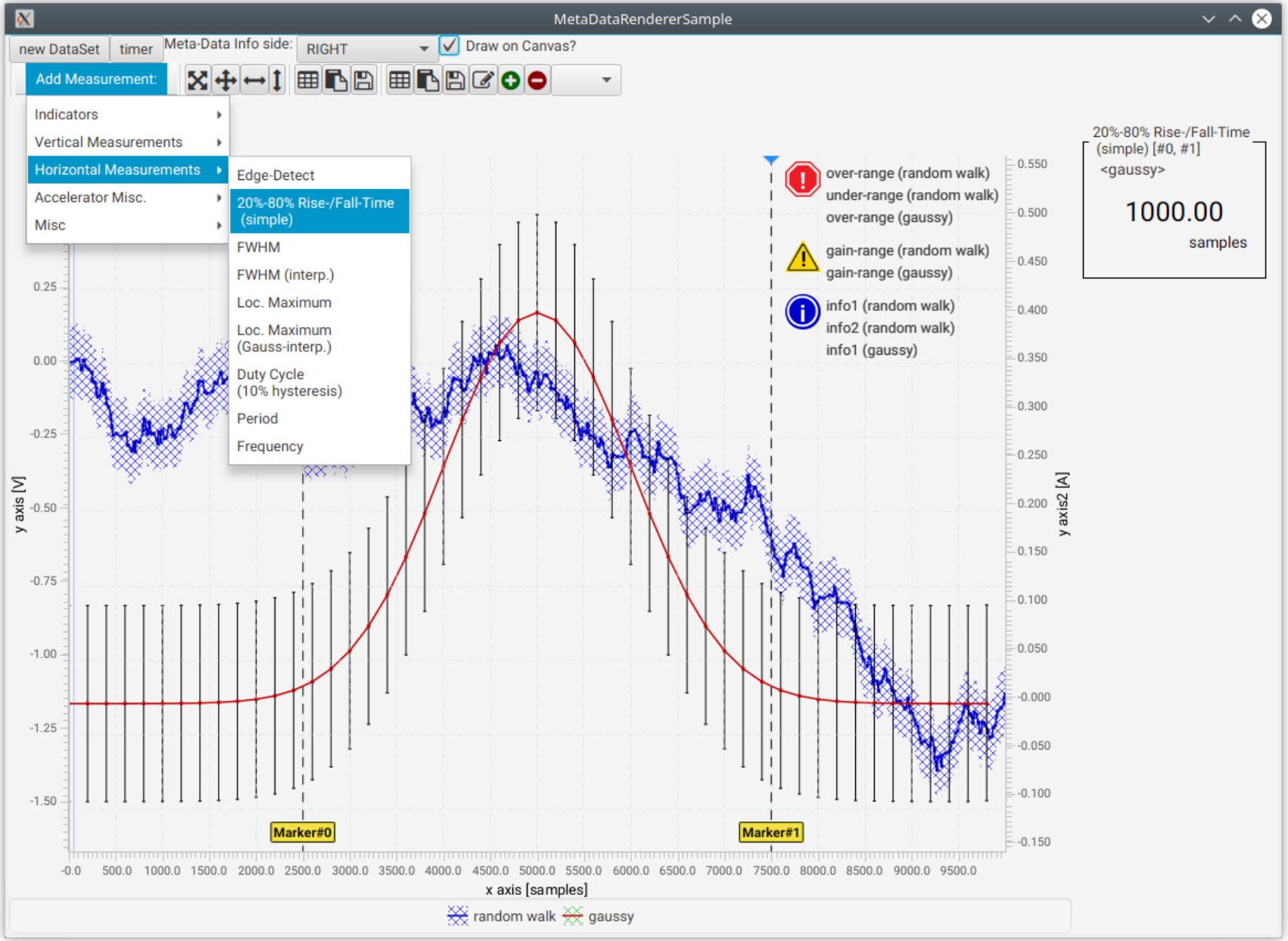
https://openjfx.io/

Principales caractéristiques de JavaFX

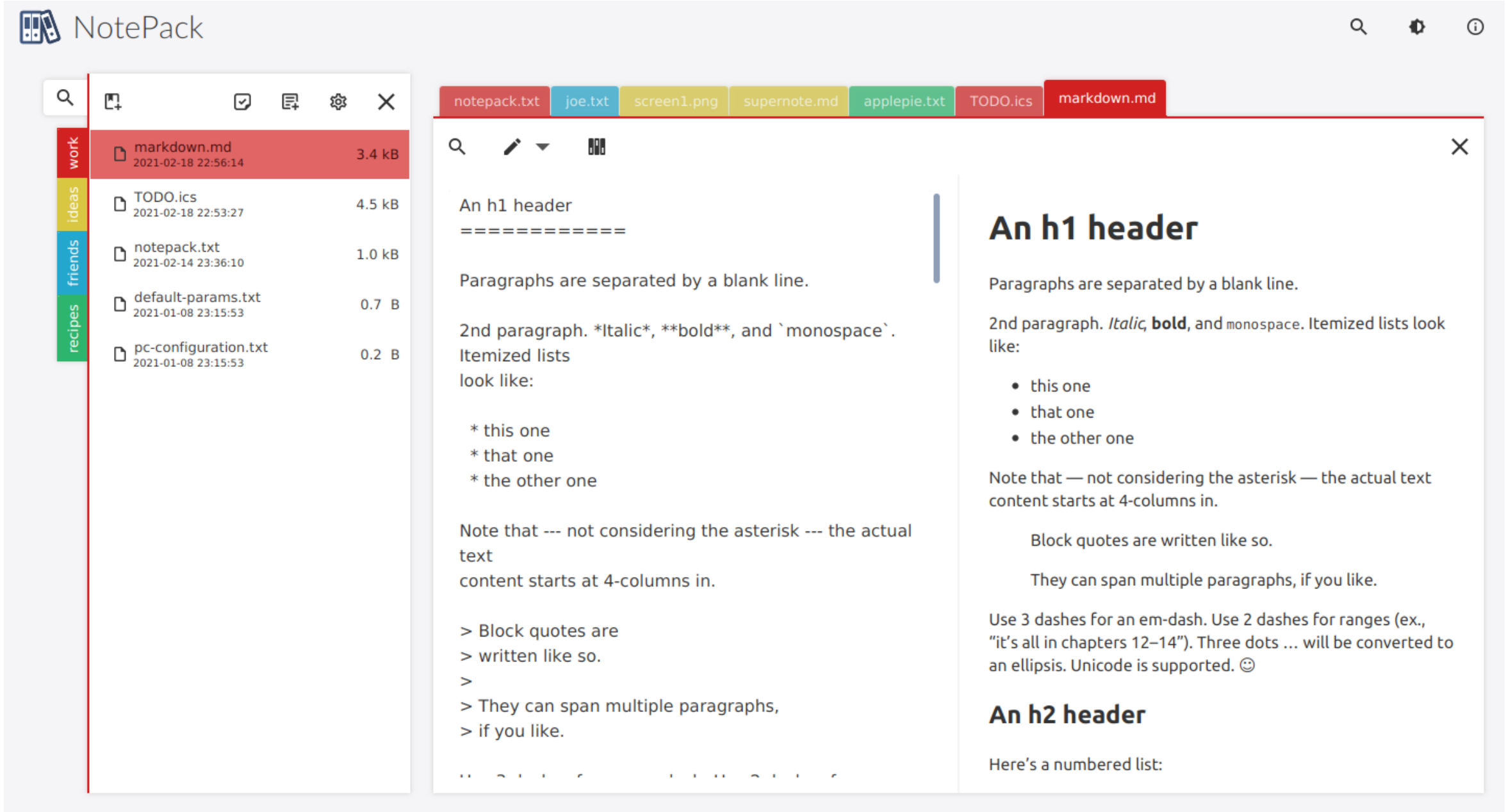
- Framework moderne pour interfaces graphiques
- Scène et graphe de nœuds
- Composants riches (UI Controls)
- Stylisation avec CSS
- FXML
- Animations et effets graphiques
- Multimédia intégré
- Graphiques 2D et 3D
- Portabilité
- Architecture orientée objet et réactive



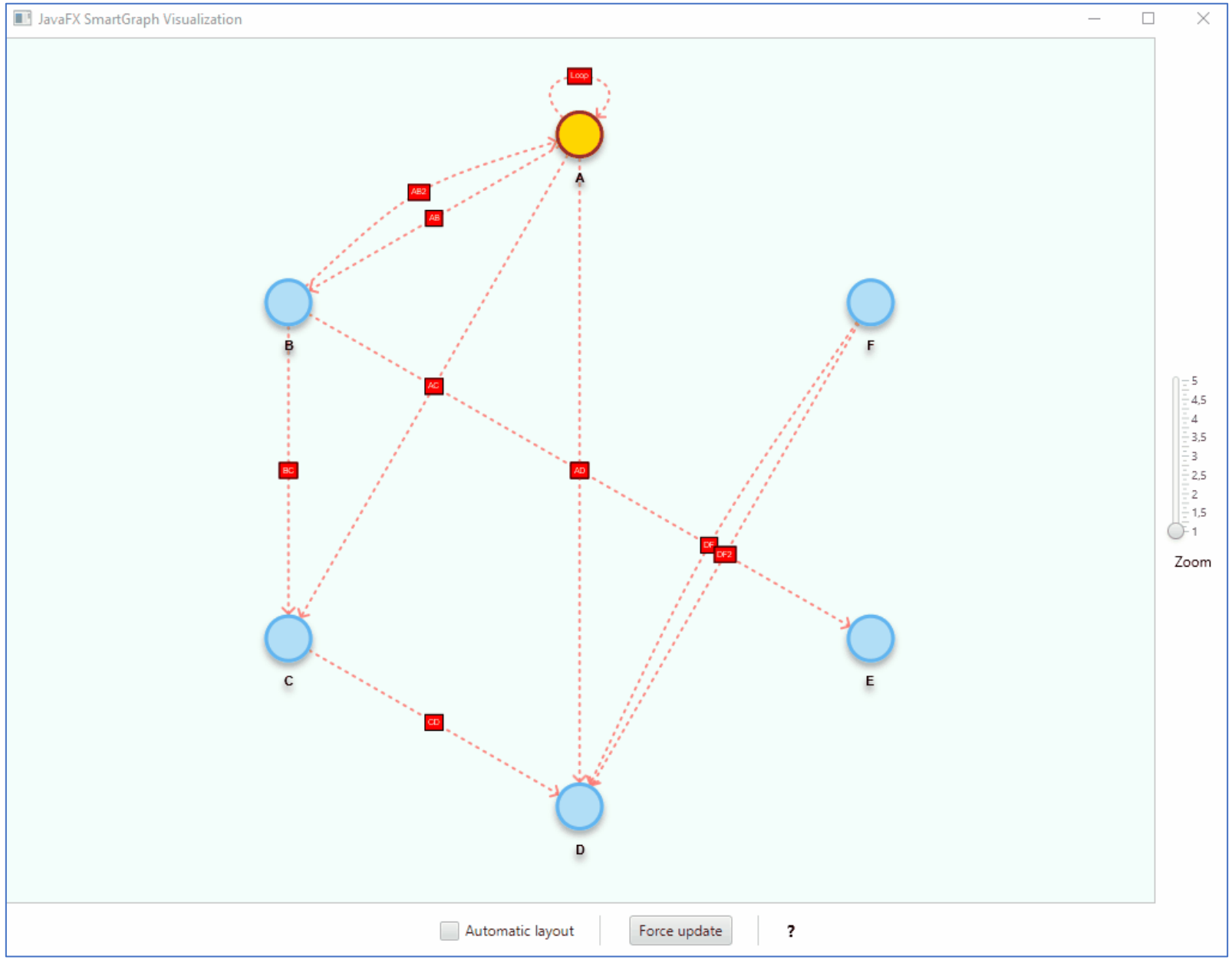
https://github.com/inforkgodara/store-pos



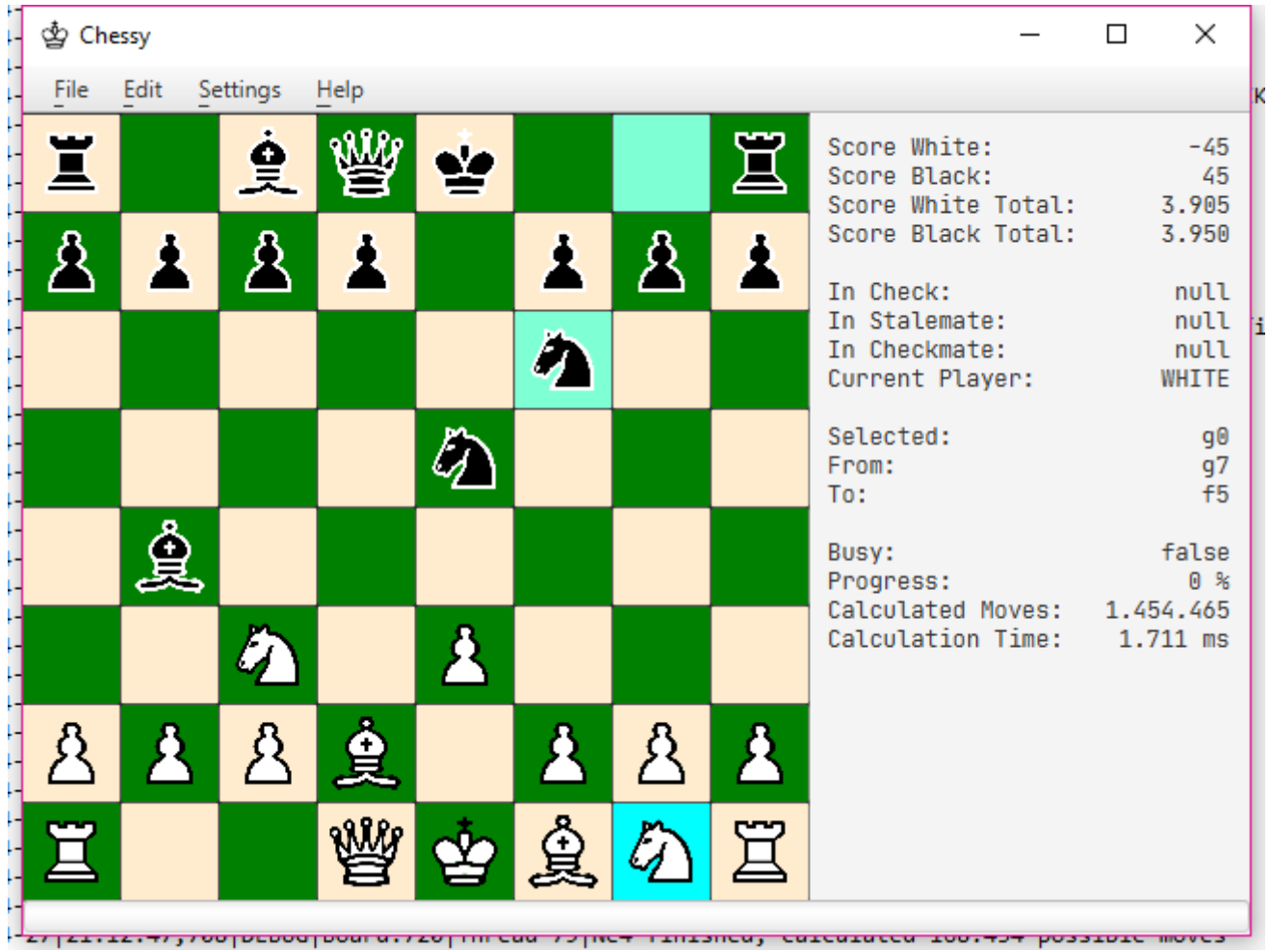
https://github.com/fair-acc/chart-fx



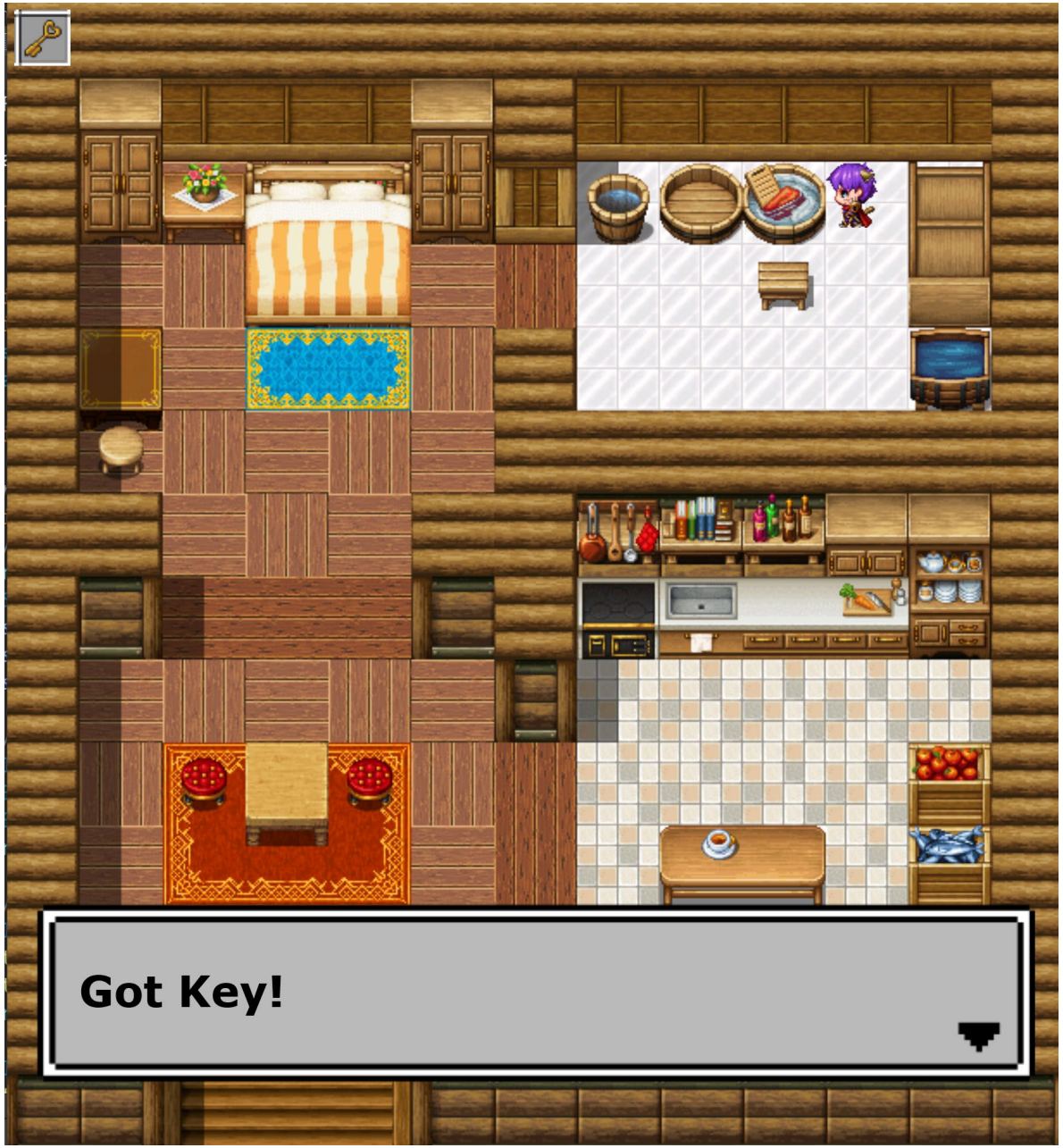
https://github.com/Ogefest/Notepack



https://github.com/brunomnsilva/JavaFXSmartGraph



https://github.com/sauce-code/chessy

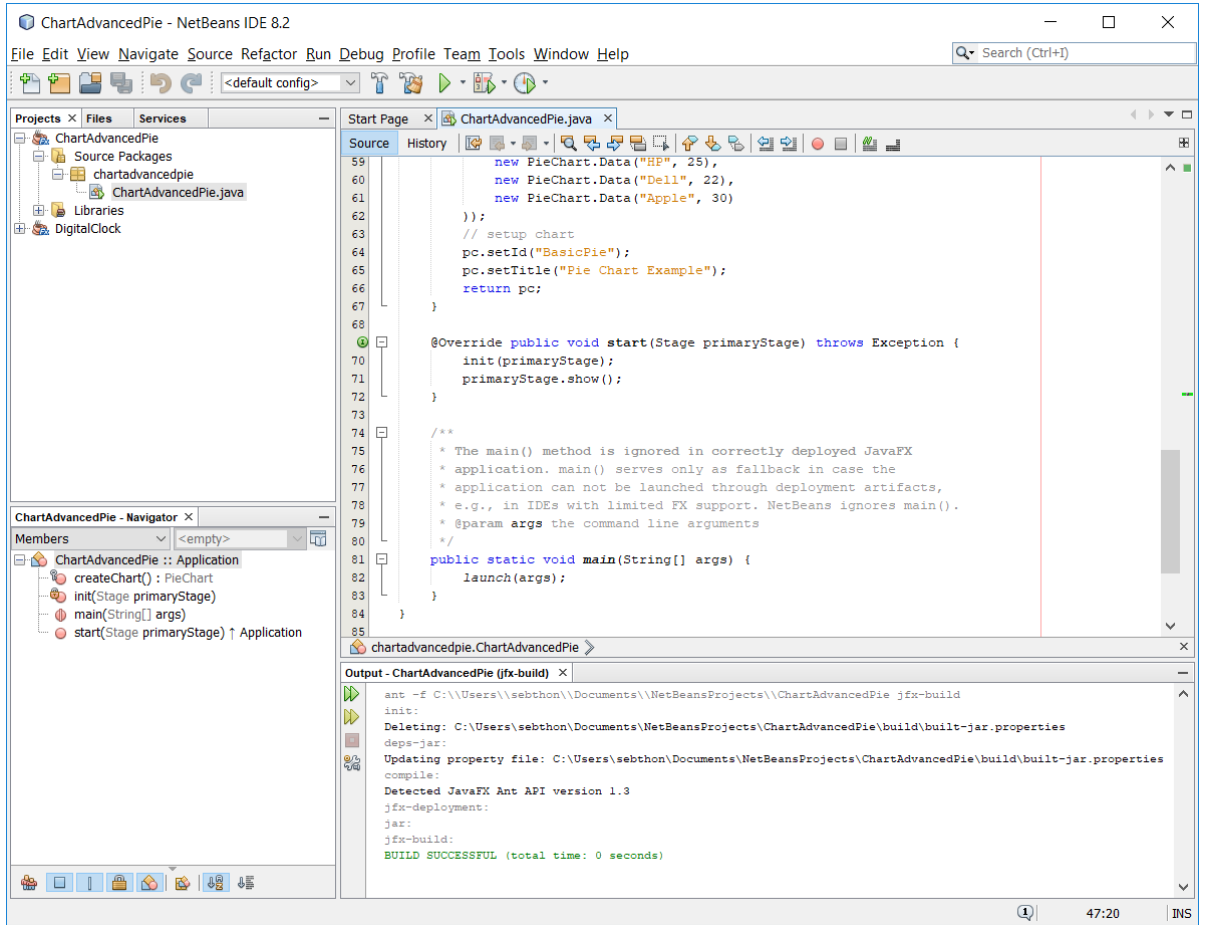


https://github.com/yongeshi/rpg-escape-game

IDE (Integrated Development Environment)

= Environnement de développement intégré.

Permet de gérer des projets comportant plusieurs fichiers, d'éditer du code, d'ajouter des packages, de compiler, de débuguer, de construire visuellement une interface graphique, etc.



Netbeans
https://netbeans.org/

Eclipse
https://www.eclipse.org/ide/

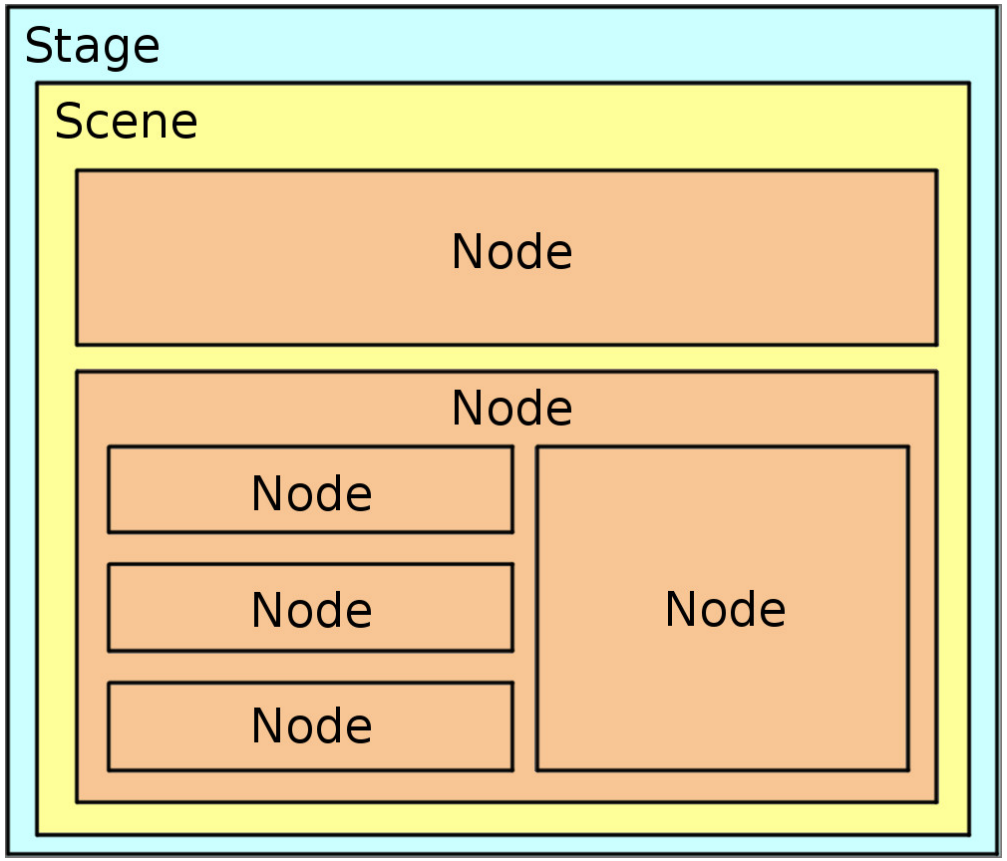
IntelliJ Idea
https://www.jetbrains.com/fr-fr/idea/

Architecture JavaFX

Stage : représente une fenêtre dans l'application (similaire à une **JFrame** en Swing). Chaque application JavaFX doit avoir au moins un Stage.

Scene : un **Stage** peut contenir une **Scene**, qui est l'arbre de nœuds représentant la structure visuelle de l'interface graphique. La scène est un conteneur pour tous les éléments graphiques (boutons, textes, images, etc.).

Nodes : les éléments individuels qui composent une scène sont appelés des "nœuds". Chaque nœud représente un objet graphique : un bouton, une image, du texte, un formulaire, etc.



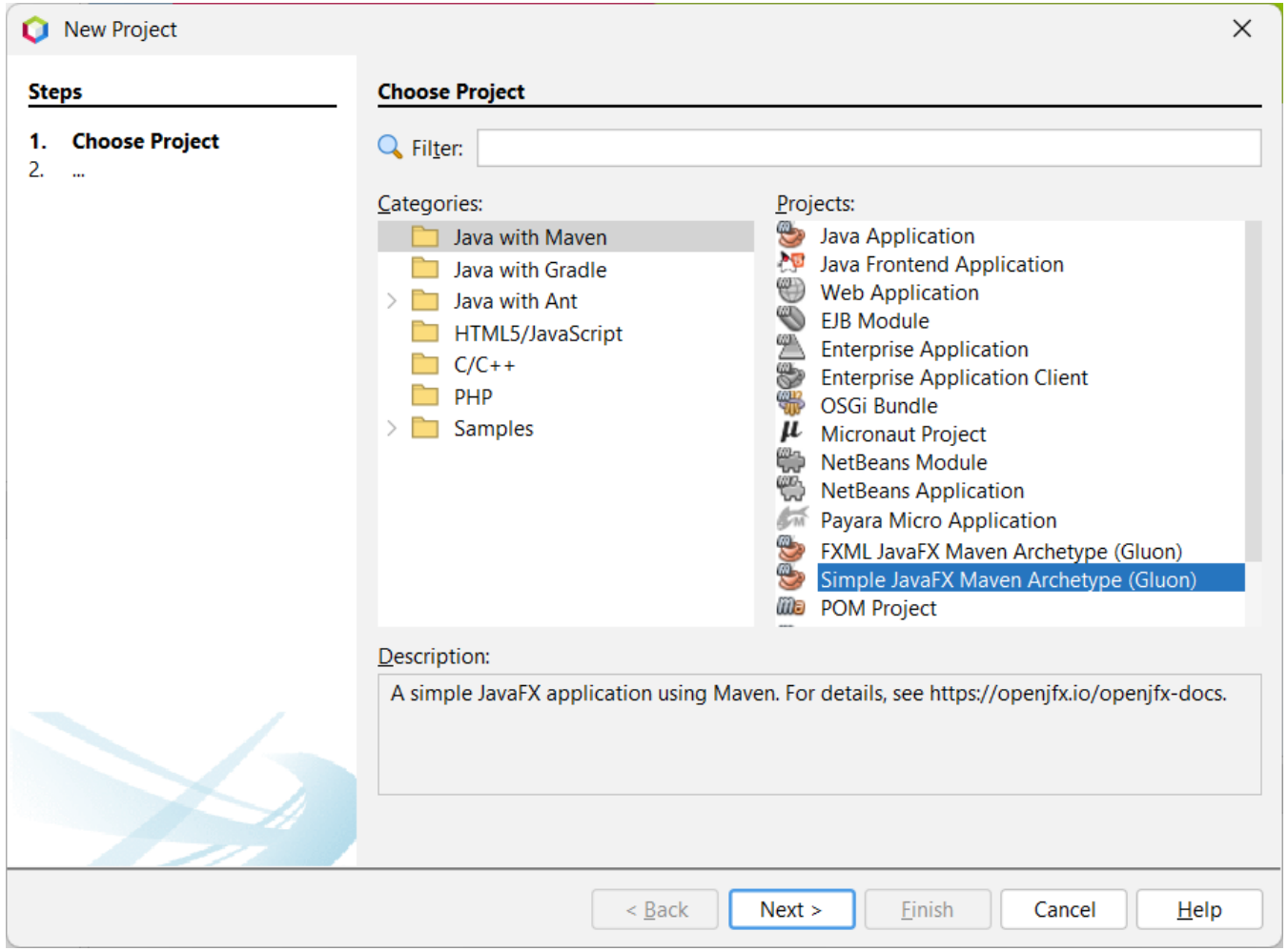
Installation et configuration de JavaFX

À partir de JDK 11, JavaFX est distribué séparément du JDK. Deux solutions :

1) Télécharger et installer le SDK JavaFX séparément.

2) Configurer JavaFX dans un IDE.

Dans l'IDE **Netbeans**, créez un projet et dans la catégorie « **Java with Maven** » choisissez « **FXML JavaFX Maven Archetype** » ou « **Simple JavaFX Maven Archetype** » .



FXML JavaFX Maven Archetype : projets JavaFX utilisant des fichiers FXML pour définir l'interface utilisateur, avec une syntaxe basée sur XML. Cela rend le code de l'interface plus facile à gérer et à séparer de la logique métier, en particulier pour des applications complexes.

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.layout.VBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.Button?>
<?import javafx.geometry.Insets?>

<VBox alignment="CENTER" spacing="20.0" xmlns="http://javafx.com/javafx/8.0.171" xmlns:fx="http://javafx.com/fxml/1"
   <children>
      <Label text="Primary View" />
      <Button fx:id="primaryButton" text="Switch to Secondary View" onAction="#switchToSecondary"/>
   </children>
   <padding>
      <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
   </padding>
</VBox>
```

Simple JavaFX Maven Archetype : projet JavaFX basique sans configuration FXML par défaut. L'interface utilisateur est directement construite en Java, avec du code JavaFX pour définir les éléments de l'UI (comme des Scene, Stage, Button, etc.). C'est plus adapté pour des projets simples ou pour ceux qui préfèrent créer l'interface via du code sans avoir besoin de fichiers FXML.

Exemple




```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
```

```
public class HelloWorld extends Application {
    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("Hello World!");

        Button btn = new Button("Fermer");
        StackPane root = new StackPane();
        root.getChildren().add(btn);

        Scene scene = new Scene(root, 300, 200);
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Explications

```
import javafx.application.Application;
```

Application : classe de base que toute application JavaFX doit étendre.

```
import javafx.stage.Stage;
```

Stage : représente une fenêtre (comme une JFrame en Swing).

```
import javafx.scene.Scene;
```

Scene : contient l'arborescence des éléments graphiques affichés dans un **Stage**.

```
import javafx.scene.control.Button;
```

Button : un bouton cliquable.

```
import javafx.scene.layout.StackPane;
```

StackPane : un conteneur (**layout**) qui empile ses enfants au centre

```
public class HelloWorld extends Application {
    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("Hello World!");

        Button btn = new Button("Fermer");
        StackPane root = new StackPane();
        root.getChildren().add(btn);

        Scene scene = new Scene(root, 300, 200);
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

La classe **HelloWorld** hérite de **Application**, ce qui permet de lancer une appli JavaFX.

start() est la méthode obligatoire qu'on doit redéfinir. Elle est appelée automatiquement quand l'application démarre. **primaryStage** est la fenêtre principale créée par JavaFX.

Crée un conteneur **StackPane**. Ajoute le bouton dans ce conteneur (au centre par défaut).

Crée une scène de 300 pixels de large et 200 pixels de haut. La scène contient **root** (le layout), donc tout son contenu (ici, le bouton).

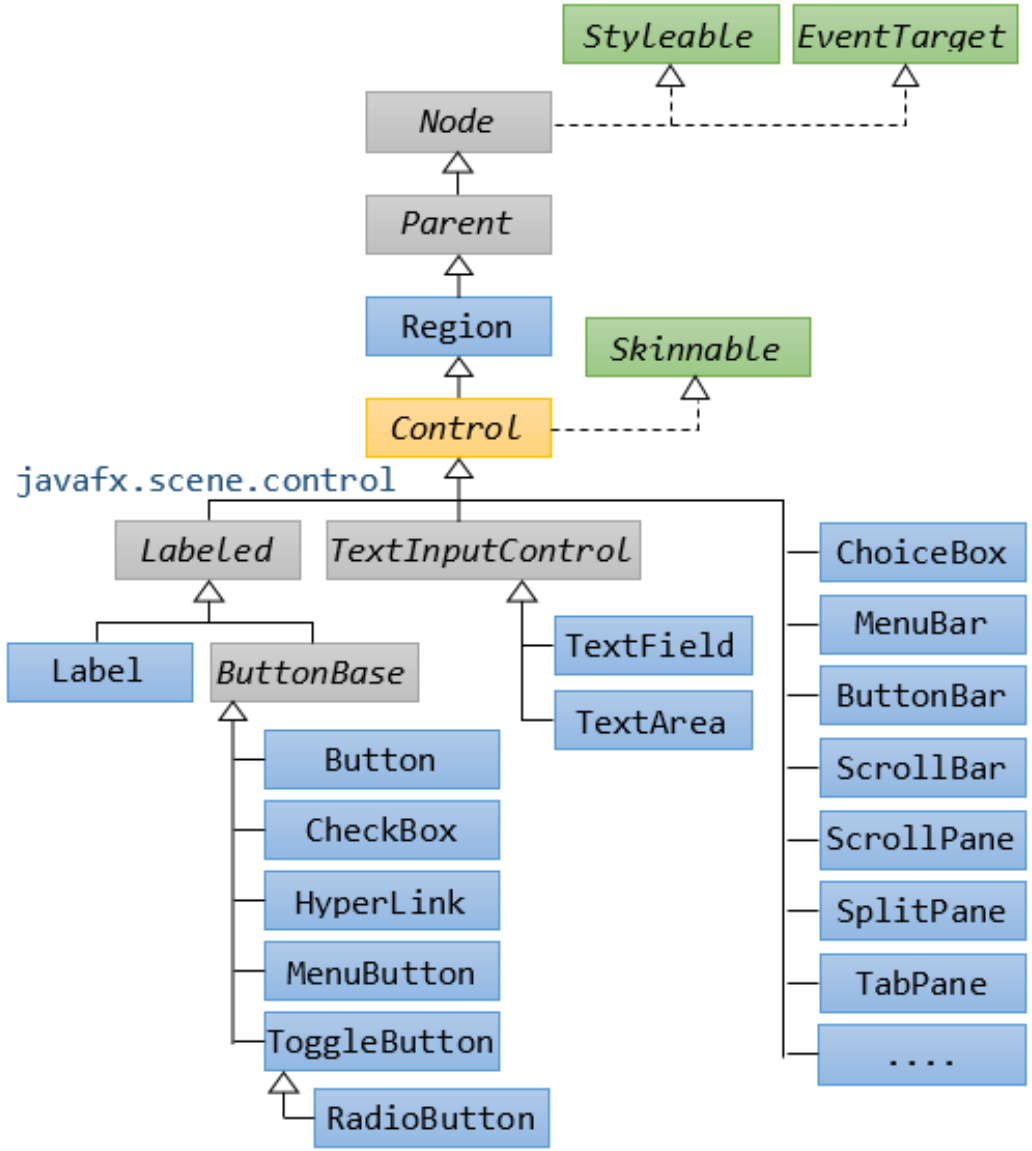
Associe la scène à la fenêtre (**Stage**).
show() affiche réellement la fenêtre.

Point d'entrée de l'application.
launch(args) est une méthode fournie par **Application** qui initialise JavaFX, puis appelle **start()**

Les composants de JavaFX

JavaFX propose une série de composants appelés "**controls**" qui sont utilisés pour créer des interfaces utilisateurs interactives.

Ces contrôles représentent des éléments comme des boutons, des champs de texte, des cases à cocher, des listes, etc.

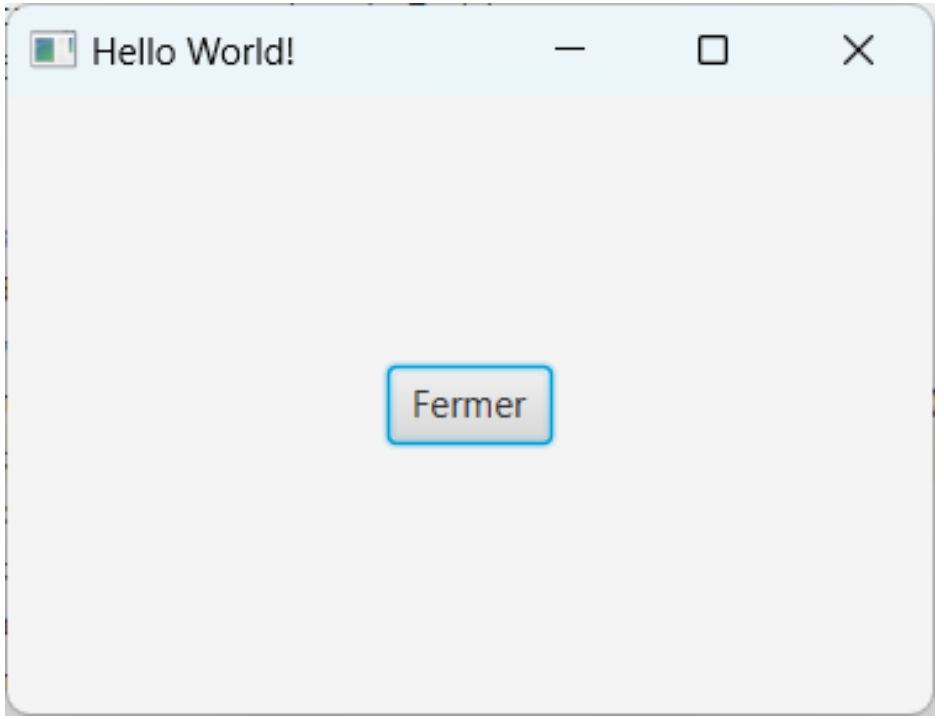


Button

https://openjfx.io/javadoc/15/javafx.controls/javafx/scene/control/Button.html

Un **Button** est un bouton sur lequel l'utilisateur peut cliquer pour déclencher une action.

```
Button btn = new Button("Fermer");
```



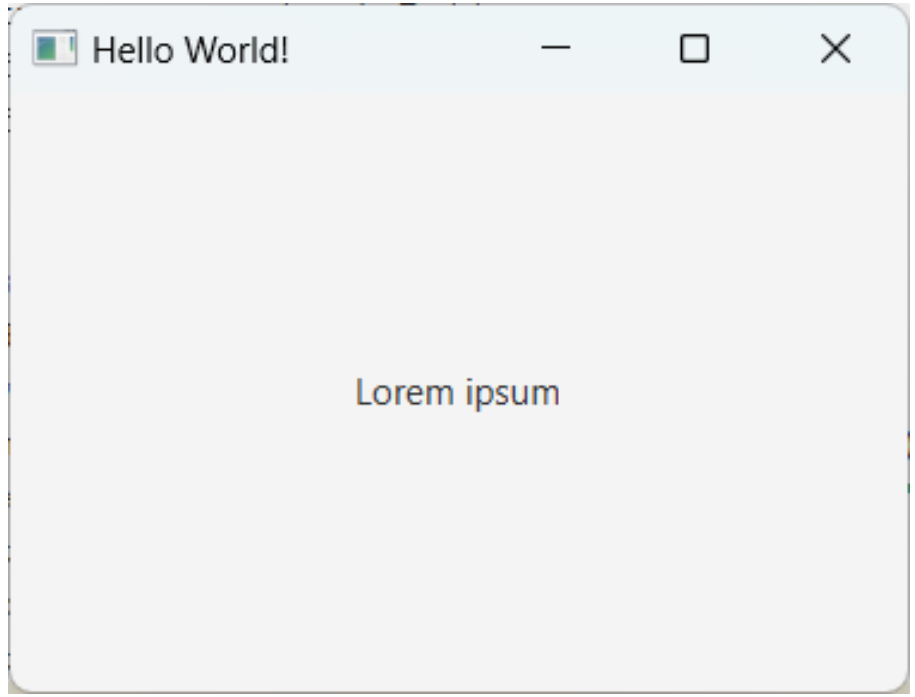
Un événement de clic sur le bouton peut être capturé avec un **EventHandler** (voir point sur la gestion d'événements).

Label

https://openjfx.io/javadoc/15/javafx.controls/javafx/scene/control/Label.html

Un **Label** est un texte statique utilisé pour afficher des informations ou des titres.

```
Label label = new Label("Lorem ipsum");
```

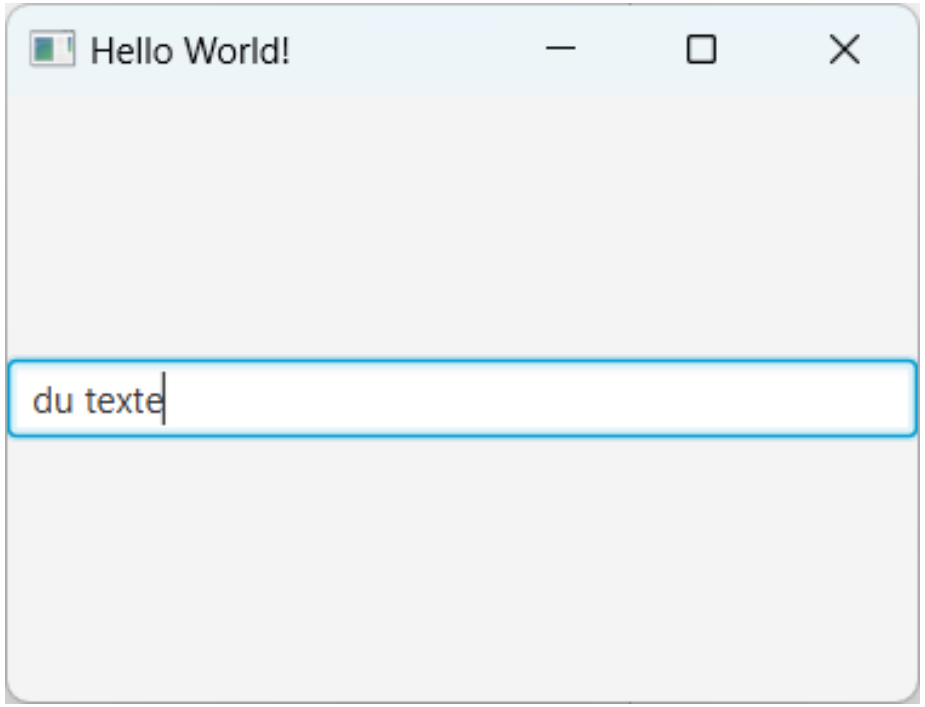


TextField

https://openjfx.io/javadoc/15/javafx.controls/javafx/scene/control/TextField.html

Un **TextField** est un champ de saisie de texte à une seule ligne.

```
TextField textField = new TextField();
```



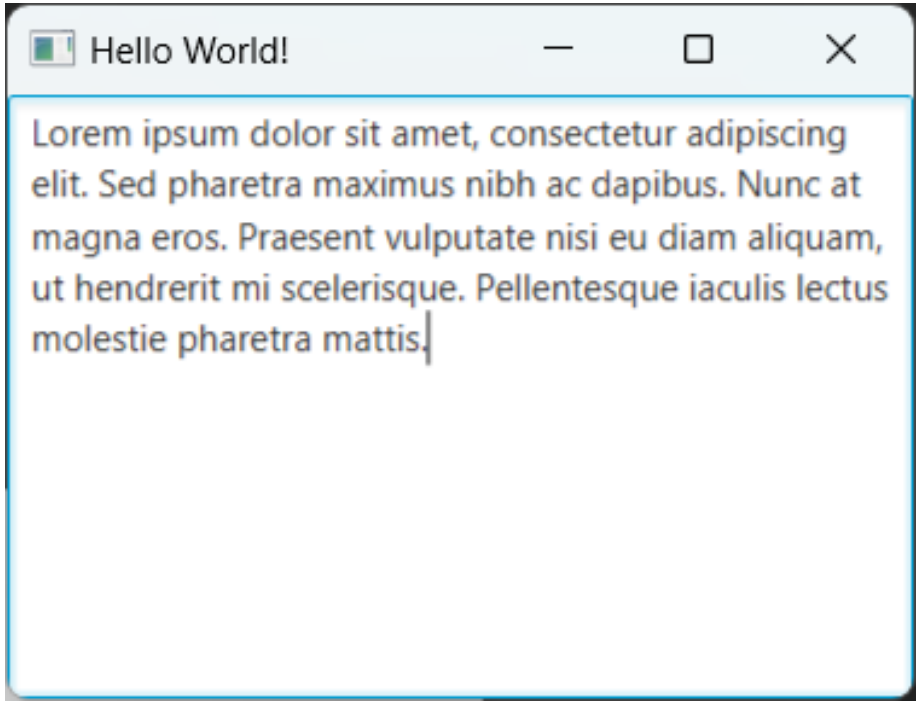
TextArea

https://openjfx.io/javadoc/15/javafx.controls/javafx/scene/control/TextArea.html

Un **TextArea** est similaire au **TextField**, mais permet la saisie sur plusieurs lignes.

```
TextArea textArea = new TextArea();
```

```
textArea.setWrapText(true);
```

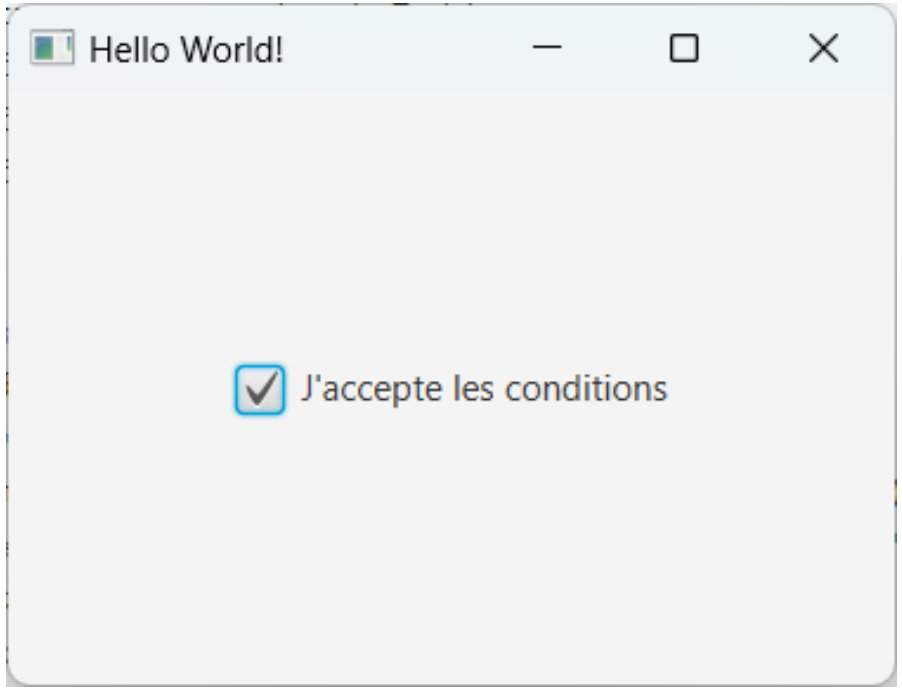


CheckBox

https://openjfx.io/javadoc/15/javafx.controls/javafx/scene/control/CheckBox.html

Un **CheckBox** permet à l'utilisateur de sélectionner ou désélectionner une option.

```
CheckBox checkBox = new CheckBox("J'accepte les conditions");
```



RadioButton

https://openjfx.io/javadoc/15/javafx.controls/javafx/scene/control/RadioButton.html

Un **RadioButton** permet à l'utilisateur de sélectionner une seule option parmi plusieurs.

```
RadioButton radio1 = new RadioButton("Option 1");
RadioButton radio2 = new RadioButton("Option 2");
```

ComboBox

https://openjfx.io/javadoc/15/javafx.controls/javafx/scene/control/ComboBox.html

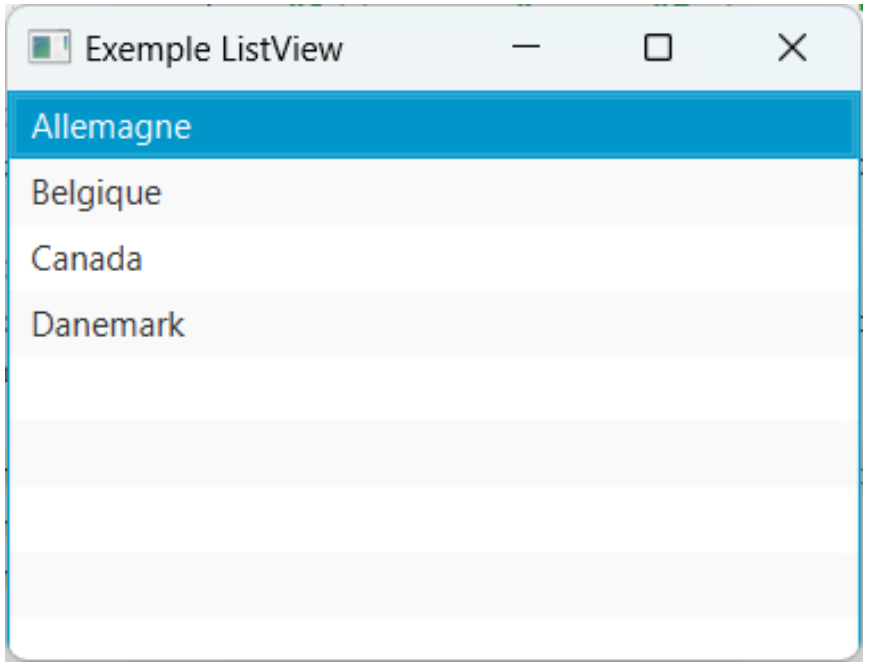
Un **ComboBox** est une liste déroulante permettant de sélectionner une option parmi plusieurs.

```
ComboBox<String> comboBox = new ComboBox<>();
comboBox.getItems().addAll("Option 1", "Option 2", "Option 3");
```

ListView

https://openjfx.io/javadoc/15/javafx.controls/javafx/scene/control/ListView.html

ListView permet d'afficher une liste simple d'éléments. Il est utile lorsque vous devez présenter une série d'éléments que l'utilisateur peut sélectionner.



```
import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.scene.Scene;
import javafx.scene.control.ListView;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class ListViewExample extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Création d'une liste observable d'éléments
        ObservableList<String> names =
                FXCollections.observableArrayList
                ("Allemagne", "Belgique", "Canada", "Danemark");

        // Création du ListView
        ListView<String> listView = new ListView<>(names);

        // Organisation de l'interface
        StackPane root = new StackPane(listView);
        Scene scene = new Scene(root, 300, 200);

        primaryStage.setTitle("Exemple ListView");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Une **ObservableList** est une liste spéciale fournie par JavaFX (javafx.collections.ObservableList).

Elle fonctionne comme une List classique (ArrayList, LinkedList…), mais elle est capable de notifier automatiquement les composants JavaFX quand son contenu change (ajout, suppression, modification).

```
ObservableList<String> names = FXCollections.observableArrayList(
    "Allemagne", "Belgique", "Canada", "Danemark"
);

ListView<String> listView = new ListView<>(names);
```

Le **ListView** est lié à la liste **names** qui contient les données (les pays). Si on modifie **names**, le **ListView** se met à jour automatiquement.

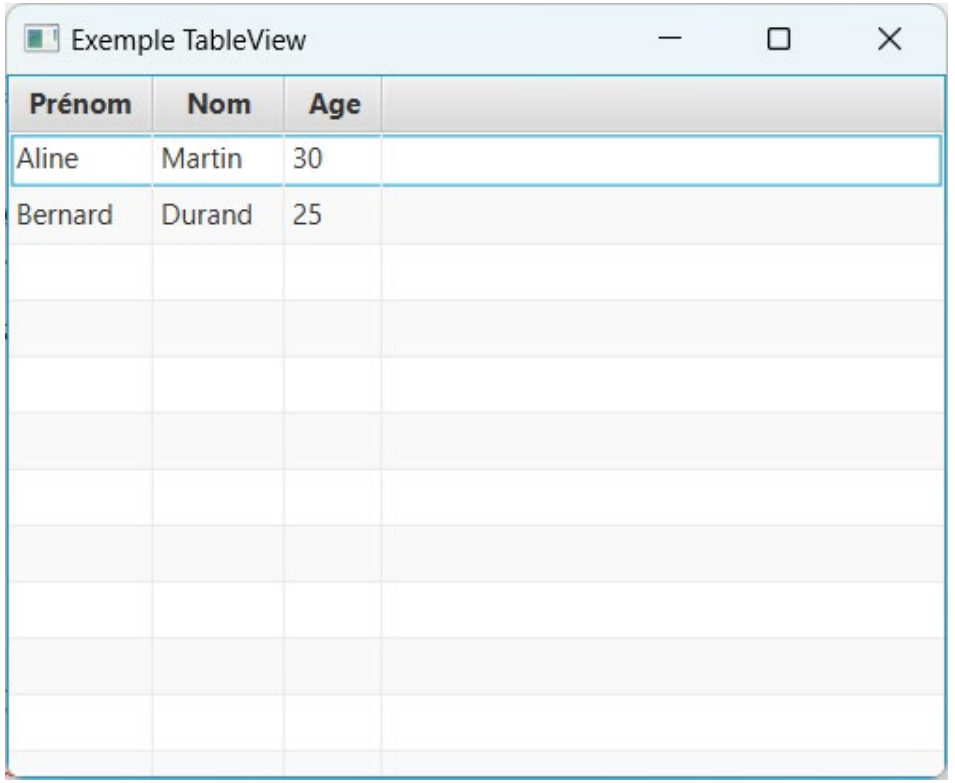
```
names.add("Espagne");        // Ajoute un élément → le ListView affiche "Espagne"
names.remove("Canada");      // Supprime un élément → "Canada" disparait de l'affichage
```

TableView

https://openjfx.io/javadoc/15/javafx.controls/javafx/scene/control/TableView.html

TableView est un composant utilisé pour afficher des données tabulaires, semblable à un tableau dans une feuille de calcul.

Il permet d'afficher des colonnes et des lignes, et chaque cellule peut contenir un objet Java.



```
public static class Person {
    private final String firstName;
    private final String lastName;
    private final int age;

    public Person(String firstName, String lastName, int age) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
    }

    public String getFirstName() {
        return firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public int getAge() {
        return age;
    }
}
```

```
public class TableViewExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        TableView<Person> table = new TableView<>();
        ObservableList<Person> data = FXCollections.observableArrayList(
                new Person("Aline", "Martin", 30),
                new Person("Bernard", "Durand", 25)
        );

        // Création des colonnes
        TableColumn<Person, String> firstNameCol = new TableColumn<>("Prénom");
        firstNameCol.setCellValueFactory(new PropertyValueFactory<>("firstName"));

        TableColumn<Person, String> lastNameCol = new TableColumn<>("Nom");
        lastNameCol.setCellValueFactory(new PropertyValueFactory<>("lastName"));

        TableColumn<Person, Integer> ageCol = new TableColumn<>("Age");
        ageCol.setCellValueFactory(new PropertyValueFactory<>("age"));

        // Ajouter les colonnes à la table
        table.getColumns().addAll(firstNameCol, lastNameCol, ageCol);
        table.setItems(data);

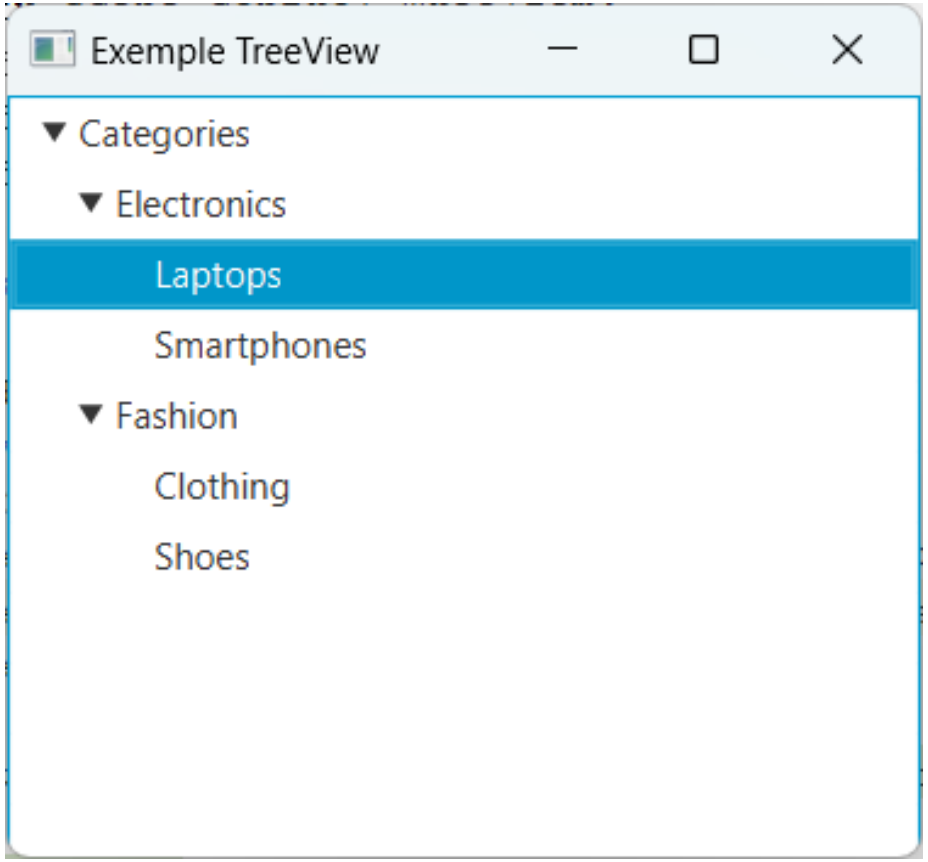
        // Organisation de l'interface
        VBox vbox = new VBox(table);
        Scene scene = new Scene(vbox, 400, 300);

        primaryStage.setTitle(« Exemple TableView");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

TreeView

TreeView est utilisé pour afficher des données hiérarchiques sous forme d'arbre, comme des répertoires et des fichiers dans un gestionnaire de fichiers.



```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.TreeItem;
import javafx.scene.control.TreeView;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class TreeViewExample extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Création des éléments de l'arbre
        TreeItem<String> rootItem = new TreeItem<>("Categories");
        TreeItem<String> electronics = new TreeItem<>("Electronics");
        TreeItem<String> fashion = new TreeItem<>("Fashion");

        rootItem.getChildren().addAll(electronics, fashion);

        electronics.getChildren().addAll(new TreeItem<>("Laptops"),
                new TreeItem<>("Smartphones"));
        fashion.getChildren().addAll(new TreeItem<>("Clothing"),
                new TreeItem<>("Shoes"));

        // Création du TreeView
        TreeView<String> treeView = new TreeView<>(rootItem);
        rootItem.setExpanded(true); // Ouvrir les éléments de l'arbre par défaut

        // Organisation de l'interface
        StackPane root = new StackPane(treeView);
        Scene scene = new Scene(root, 300, 250);

        primaryStage.setTitle("Exemple TreeView");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Disposition des composants avec des layouts

Les composants de JavaFX doivent être organisés visuellement dans l'interface. JavaFX propose plusieurs systèmes de disposition appelés layouts, qui permettent d'arranger les éléments graphiques en fonction de la structure souhaitée.

Principaux layouts : HBox, VBox, GridPane, BorderPane, StackPane, FlowPane…

HBox

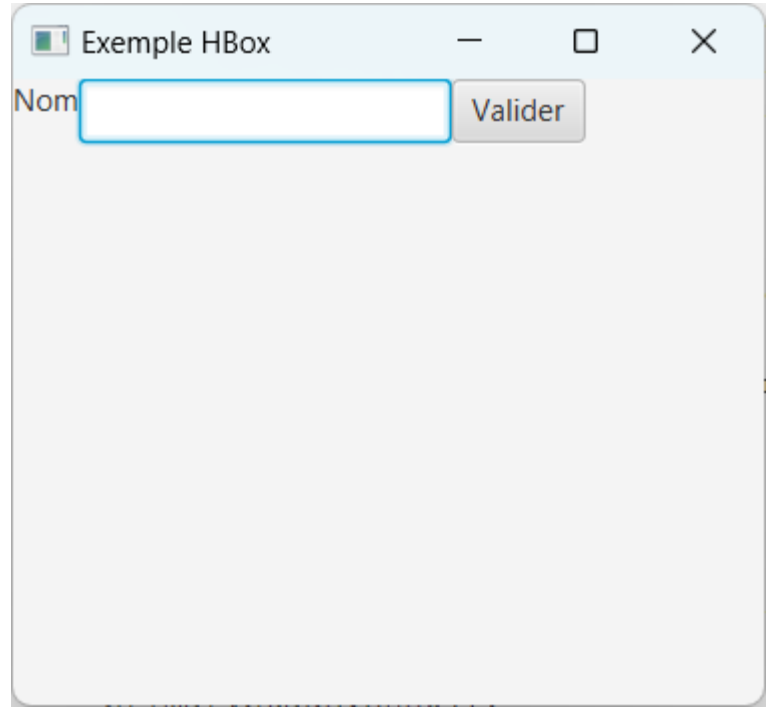
https://openjfx.io/javadoc/24/javafx.graphics/javafx/scene/layout/HBox.html

Un HBox (Horizontal Box) dispose les éléments enfants horizontalement, dans une seule ligne.

```
Label label = new Label("Nom");
TextField textfield = new TextField();
Button button = new Button("Valider");

HBox hbox = new HBox();
hbox.getChildren().addAll(label, textField, button);
Scene scene = new Scene(hbox, 300, 250);

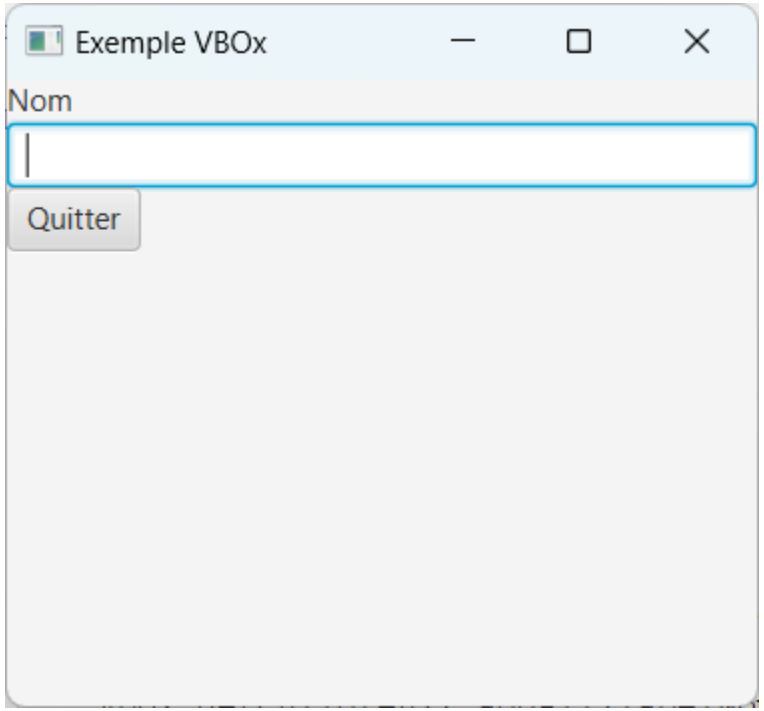
primaryStage.setTitle("Exemple HBox");
primaryStage.setScene(scene);
primaryStage.show();
```



VBox

https://openjfx.io/javadoc/24/javafx.graphics/javafx/scene/layout/VBox.html

Un **VBox** (Vertical Box) dispose les éléments enfants verticalement, dans une seule colonne.

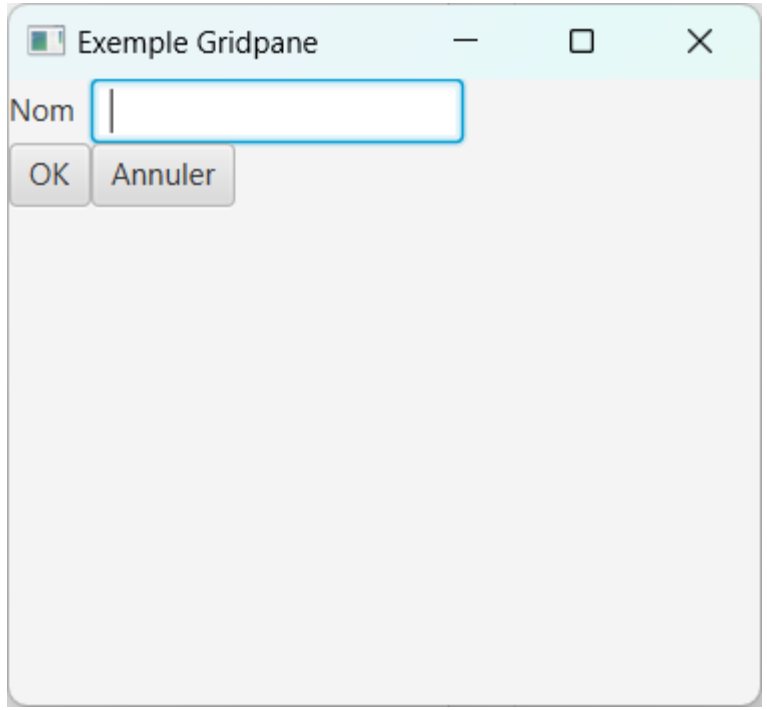



```
VBox vbox = new VBox();
vbox.getChildren().addAll(label, textField, button);
```

GridPane

Un **GridPane** permet de disposer les éléments dans une grille à deux dimensions, avec des lignes et des colonnes.

```
GridPane gridPane = new GridPane();
gridPane.add(labelNom, 0, 0);
gridPane.add(textfieldNom, 1, 0);
gridPane.add(buttonOK, 0, 1);
gridPane.add(buttonAnnuler, 1, 1);
```

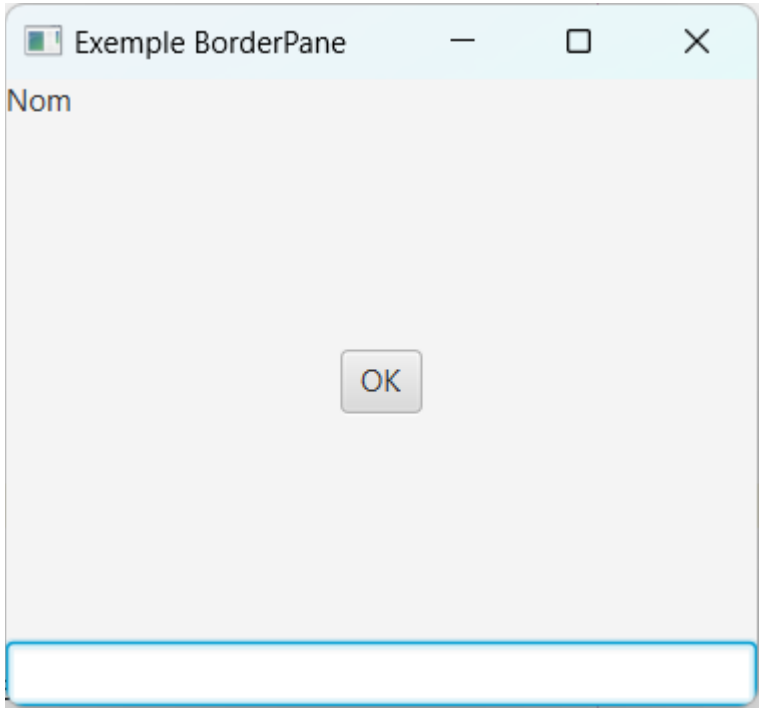


Utilisé pour des formulaires ou des dispositions complexes nécessitant plusieurs rangées et colonnes.

BorderPane

Un **BorderPane** dispose les éléments dans cinq régions principales : haut, bas, gauche, droite et centre.

```
BorderPane borderPane = new BorderPane();
borderPane.setTop(label);
borderPane.setCenter(button);
borderPane.setBottom(textField);
```



Utilisé pour une interface structurée en parties distinctes (par exemple un en-tête, un pied de page, un panneau latéral, etc.).

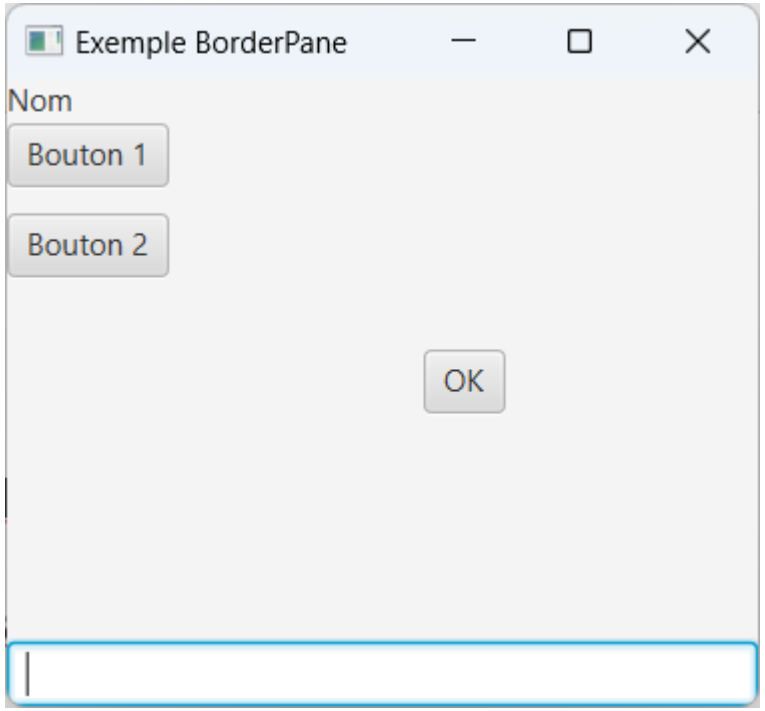
BorderPane

On peut mettre des layouts dans des layouts, par exemple ici on ajoute un layout **VBox** avec 2 boutons dans la partie gauche du layout **BorderPane**.

```
VBox leftBox = new VBox();
Button btn1 = new Button("Bouton 1");
Button btn2 = new Button("Bouton 2");

// Ajout des boutons dans la VBox
leftBox.getChildren().addAll(btn1, btn2);

// Ajout de la VBox dans la partie gauche du BorderPane
borderPane.setLeft(leftBox);
```



StackPane

Un **StackPane** empile les éléments les uns sur les autres, comme des couches.

```
StackPane stackPane = new StackPane();
stackPane.getChildren().addAll(rectangle, button);
```

Utilisé pour des superpositions simples (par exemple un bouton sur une image).

FlowPane

Un **FlowPane** dispose les éléments de manière séquentielle, en les remplissant dans une ligne ou une colonne, et en créant des lignes supplémentaires si nécessaire.

```
FlowPane flowPane = new FlowPane();
flowPane.getChildren().addAll(button1, button2, button3);
```

Gestion des événements

JavaFX suit un modèle de gestion d'événements où les interactions utilisateur (clics, frappes au clavier, mouvements de souris, etc.) sont capturées et traitées par des gestionnaires d'événements (**EventHandler**).

Evénements de clic sur un bouton

```
bouton.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        System.out.println("Bouton cliqué!");
    }
});
```

Ou plus simplement avec une **expression lambda** :

```
bouton.setOnAction(event -> System.out.println("Bouton cliqué!"));
```

Une seule instruction :

```
bouton.setOnAction(event -> System.out.println("Bouton cliqué!"));
```

Plusieurs instructions :

```
bouton.setOnAction(event -> {
        System.out.println("Bouton cliqué!");
        System.out.println("Autre instruction");
});
```

Expression Lambda

Les **expressions lambda** sont une manière courte et lisible d'écrire du code qui représente une fonction.

Sans expression lambda :

```
Button btn = new Button("Cliquez-moi");

// Version avant Java 8 (classe anonyme)
btn.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        System.out.println("Bouton cliqué !");
    }
});
```

Avec une expression Lambda :

```
btn.setOnAction(event -> System.out.println("Bouton cliqué !"));
```

Une lambda a la forme :

(paramètres) -> { corps }

paramètres : les arguments (comme dans une méthode).

-> : opérateur lambda ("devient").

corps : le code à exécuter.

Exemples :

```
() -> System.out.println("Hello")        // aucun paramètre
x -> x * x                               // un paramètre
(a, b) -> a + b                          // plusieurs paramètres
(x, y) -> {                              // bloc avec plusieurs instructions
    int sum = x + y;
    return sum;
}
```

MouseEvent (Evénements de souris)

Les événements de souris permettent de capturer des actions comme le clic, le déplacement ou le survol de la souris :

setOnMouseClicked : Lorsqu'un utilisateur clique sur un élément.

setOnMouseEntered : Lorsqu'une souris entre dans la zone d'un élément.

setOnMouseExited : Lorsqu'une souris quitte la zone d'un élément.

Exemple : Changer le texte d’un bouton au survol de la souris

```
button.setOnMouseEntered(event -> button.setText("Souris sur le bouton"));
button.setOnMouseExited(event -> button.setText("Click Me"));
```

KeyEvent (Événements clavier)

Les événements de clavier permettent de capturer les touches pressées par l'utilisateur :

setOnKeyPressed : Lorsqu'une touche est enfoncée.

setOnKeyReleased : Lorsqu'une touche est relâchée.

setOnKeyTyped : Lorsqu'une touche est tapée (enfoncée puis relâchée).

Exemple : Détecter la touche 'Entrée' dans un champ de texte :

```
TextField textField = new TextField();
textField.setOnKeyPressed(event -> {
    if (event.getCode() == KeyCode.ENTER) {
        System.out.println("Entrée pressée : " + textField.getText());
    }
});
```

Gestion avancée des événements

Chaînage d'événements :

Vous pouvez créer une chaîne de gestionnaires d'événements pour réagir à plusieurs types d'événements sur le même composant.

Exemple : Gérer à la fois un clic et un survol de souris sur un bouton :

```
Button btn = new Button("Click or Hover me!");
btn.setOnMouseClicked(event -> System.out.println("Bouton cliqué!"));
btn.setOnMouseEntered(event -> btn.setText("Mouse Over!"));
btn.setOnMouseExited(event -> btn.setText("Click or Hover me!"));
```

Gestion avancée des événements

Gestion des événements personnalisés :

En plus des événements standards, JavaFX permet de créer des événements personnalisés. Cela peut être utile si vous avez des événements spécifiques à gérer dans des applications complexes.

```
EventType<MyEvent> MY_EVENT =
        new EventType<>(Event.ANY, "MY_EVENT");

public class MyEvent extends Event {
    public static final EventType<MyEvent> MY_CUSTOM_EVENT
            = new EventType<>(Event.ANY, "MY_CUSTOM_EVENT");

    public MyEvent() {
        super(MY_CUSTOM_EVENT);
    }
}
```

```
// Utilisation dans un composant
Node node = new Node();
node.addEventHandler(MyEvent.MY_CUSTOM_EVENT,
event -> System.out.println("Événement personnalisé
capturé!"));
```

Gestion avancée des événements

Propagation des événements :

Lorsqu’un événement est capturé, il se propage dans l’arborescence de nœuds. Cette propagation peut être contrôlée.

Capture des événements : Si vous souhaitez intercepter un événement avant qu'il n'atteigne sa cible, vous pouvez utiliser la capture des événements.

Consommation des événements : Vous pouvez marquer un événement comme consommé pour empêcher sa propagation.

```
button.setOnAction(event -> {
    System.out.println("Action capturée");
    event.consume();        // Empêche l'événement de continuer à se propager
});
```

Binding et propriétés

Le binding est un mécanisme qui permet de synchroniser les valeurs des propriétés entre plusieurs composants d'une application, facilitant ainsi la gestion des données et des interfaces dynamiques.

Le binding est un outil puissant qui permet de relier automatiquement deux ou plusieurs propriétés afin qu'un changement dans l'une entraîne automatiquement une mise à jour dans l'autre. Cela permet de réduire la complexité du code et d'éviter des mises à jour manuelles répétitives.

Propriétés

Une **propriété** dans JavaFX est une valeur associée à un objet qui peut être observée pour détecter les changements. Ces propriétés sont dites ***observables***, ce qui signifie que lorsqu'elles sont modifiées, tous les objets qui les surveillent (appelés **listeners**) sont notifiés.

JavaFX offre une série de classes pour manipuler des propriétés :

SimpleStringProperty : Représente une chaîne de caractères.

SimpleIntegerProperty : Représente un integer.

SimpleDoubleProperty : Représente un double.

SimpleBooleanProperty : Représente un booléen.

…

Propriétés

Les propriétés sont utiles pour :

- **Suivre les changements d'état** : Si un composant a une propriété qui change souvent (comme une valeur numérique ou un texte), une propriété observable permet de réagir automatiquement à ces changements sans avoir à surveiller manuellement l’état de chaque composant.

- **Synchroniser les données** : Lorsque deux composants (comme un champ de texte et un label) doivent partager la même valeur, les propriétés permettent de maintenir cette synchronisation.

Propriétés

Exemple : chaque fois que la propriété name est modifiée, le listener est notifié et affiche l'ancien et le nouveau nom.

```
SimpleStringProperty name = new SimpleStringProperty("Pierre");

name.addListener((observable, oldValue, newValue) -> {
   System.out.println("Nom modifié : " + oldValue + " -> " + newValue);
});

// Modifier la valeur de la propriété
name.set("Paul");
```

Dans l'expression lambda (observable, oldValue, newValue), le paramètre **observable** représente la propriété observée. Ici, il s'agit de **name**, l'instance de SimpleStringProperty. Cela permet d'accéder à la propriété source de la notification si nécessaire.

Binding unidirectionnel

Le binding unidirectionnel relie une propriété à une autre dans une seule direction. Si la première propriété change, la seconde est automatiquement mise à jour. Cependant, le changement de la seconde propriété n'affecte pas la première.

Exemple : un Label affiche automatiquement le texte entré dans un TextField grâce au binding unidirectionnel, sans avoir besoin de code supplémentaire :

```
TextField textField = new TextField();
Label label = new Label();

// Liaison unidirectionnelle entre le texte du TextField et le texte du Label
label.textProperty().bind(textField.textProperty());
```

Binding bidirectionnel

Le binding bidirectionnel permet de lier deux propriétés de manière réciproque. Si l'une change, l'autre est mise à jour, et vice versa. Cela est particulièrement utile dans les interfaces où l'utilisateur peut interagir avec les deux composants, comme un TextField et un Slider.

Exemple : un Slider et un TextField sont liés de manière bidirectionnelle. Ici, le NumberStringConverter est utilisé pour convertir les valeurs numériques du Slider en texte et vice versa. :

```
Slider slider = new Slider(0, 100, 50);
TextField textField = new TextField();

// Liaison bidirectionnelle entre le Slider et le TextField
textField.textProperty().bindBidirectional(slider.valueProperty(), new NumberStringConverter());
```

Utilisation des propriétés observables dans des objets

Exemple :

Création d'une classe Person avec **des propriétés observables** pour les champs name et age.

Cette classe peut ensuite être utilisée dans une interface JavaFX pour observer les changements de valeurs de name et age et mettre à jour l'interface en conséquence.

```
public class Person {
    private SimpleStringProperty   name;
    private SimpleIntegerProperty age;
```

```
public class Person {
   private SimpleStringProperty name
         = new SimpleStringProperty(this, "name", "");
   private SimpleIntegerProperty age
         = new SimpleIntegerProperty(this, "age", 0);

   public Person(String name, int age) {
      this.name.set(name);
      this.age.set(age);
   }

   public String getName() {
      return name.get();
   }

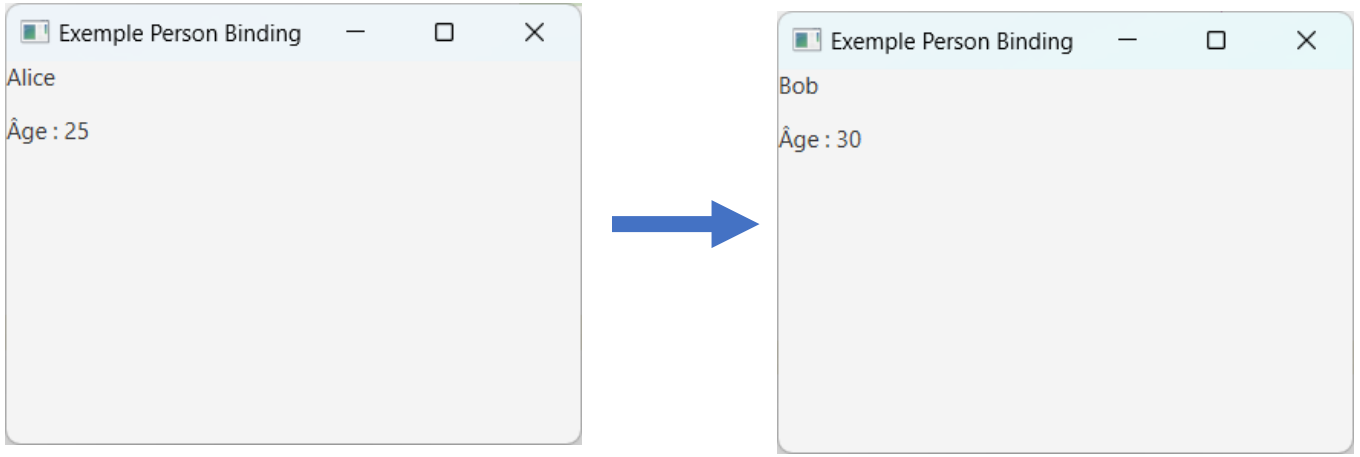
   public void setName(String name) {
      this.name.set(name);
   }

   public SimpleStringProperty nameProperty() {
      return name;
   }

   public int getAge() {
      return age.get();
   }

   public void setAge(int age) {
      this.age.set(age);
   }

   public SimpleIntegerProperty ageProperty() {
      return age;
   }
}
```




```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class PersonBinding extends Application {
    @Override
    public void start(Stage primaryStage) {
        Person person = new Person("Alice", 25);

        // Label lié à la propriété name
        Label nameLabel = new Label();
        nameLabel.textProperty().bind(person.nameProperty());

        // Label lié à la propriété age
        Label ageLabel = new Label();
        ageLabel.textProperty().bind(person.ageProperty()
                                     .asString("Âge : %d"));
        VBox root = new VBox(10, nameLabel, ageLabel);
        Scene scene = new Scene(root, 300, 200);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Exemple de binding");
        primaryStage.show();

        // mise à jour de la personne se répercutera
        // dans les labels de l'interface
        person.setName("Bob");
        person.setAge(30);
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Applications multi-fenêtres

Dans de nombreuses applications, il est courant de devoir ouvrir et fermer des fenêtres pour différentes sections de l'application (par exemple, une fenêtre de paramètres, une fenêtre d'information, etc.).

Dans JavaFX, une application peut avoir plusieurs fenêtres (instance de la classe **Stage**). Il existe au moins un Stage principal, appelé **primaryStage**, créé par défaut lors du démarrage de l’application

Chaque fenêtre peut contenir sa propre **Scene** (hiérarchie d'objets graphiques).

Applications multi-fenêtres

Deux possibilités pour naviguer entre plusieurs fenêtres dans une application :

1) Créer une nouvelle instance de la classe **Stage** et lui associer une **Scene** différente.

2) Alterner entre plusieurs **Scene** dans un même **Stage**.

1) Pour ouvrir une nouvelle fenêtre dans JavaFX, vous pouvez créer une nouvelle instance de la classe Stage et lui associer une Scene différente.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class MultiWindowExample extends Application {

    @Override
    public void start(Stage primaryStage) {
        Button openWindowButton = new Button("Open New Window");

        openWindowButton.setOnAction(event -> openNewWindow());

        StackPane root = new StackPane(openWindowButton);
        Scene scene = new Scene(root, 300, 200);

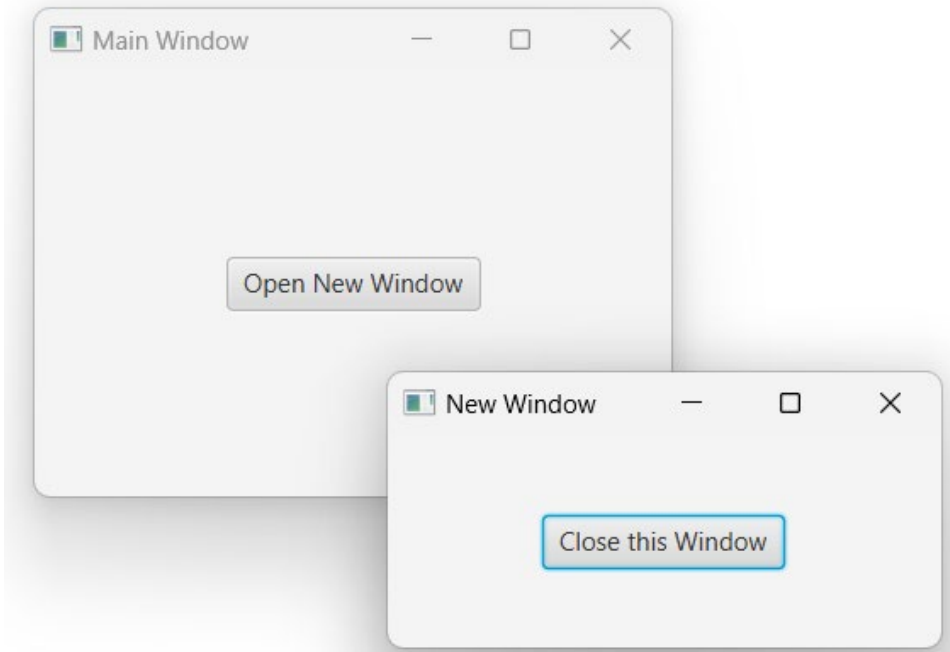
        primaryStage.setTitle("Main Window");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    // Méthode pour ouvrir une nouvelle fenêtre
    private void openNewWindow() {
        // Créer une nouvelle fenêtre (Stage)
        Stage newWindow = new Stage();
        Button closeButton = new Button("Close this Window");

        // Fermer la nouvelle fenêtre
        closeButton.setOnAction(event -> newWindow.close());
        StackPane root = new StackPane(closeButton);
        Scene scene = new Scene(root, 200, 100);
        newWindow.setScene(scene);

        newWindow.setTitle("New Window");
        newWindow.show();           // Afficher la nouvelle fenêtre
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```


Main Window
Open New Window
New Window
Close this Window

2) Alterner entre plusieurs Scenes dans un même Stage

Dans certaines applications, il peut être plus pratique de changer de Scene plutôt que de créer une nouvelle Stage. Cela est utile pour des applications où une seule fenêtre est utilisée, mais avec différentes interfaces (par exemple, un écran de connexion suivi d'un écran de tableau de bord).

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class SceneSwitchExample extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Première scène avec un bouton pour passer à la deuxième scène
        Button goToScene2Button = new Button("Go to Scene 2");
        StackPane scene1Root = new StackPane(goToScene2Button);
        Scene scene1 = new Scene(scene1Root, 300, 200);

        // Deuxième scène avec un bouton pour revenir à la première scène
        Button goToScene1Button = new Button("Go back to Scene 1");
        StackPane scene2Root = new StackPane(goToScene1Button);
        Scene scene2 = new Scene(scene2Root, 300, 200);

        // Gestion de la navigation entre les scènes
        goToScene2Button.setOnAction(event -> primaryStage.setScene(scene2));
        goToScene1Button.setOnAction(event -> primaryStage.setScene(scene1));

        // Configuration et affichage de la scène initiale
        primaryStage.setTitle("Scene Switching Example");

        // Affichage de la première scène au démarrage
        primaryStage.setScene(scene1);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Passer des données entre fenêtres

Il est souvent nécessaire de passer des données d'une fenêtre à une autre. Cela peut être fait en utilisant des paramètres dans les constructeurs ou en passant des objets comme arguments lors de la création d'une nouvelle fenêtre.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class DataPassingExample extends Application {

    @Override
    public void start(Stage primaryStage) {
        Button openWindowButton = new Button("Open New Window");

        openWindowButton.setOnAction(
                event -> openNewWindow("Hello from Main Window"));

        StackPane root = new StackPane(openWindowButton);
        Scene scene = new Scene(root, 300, 200);

        primaryStage.setTitle("Main Window");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    // Méthode pour ouvrir une nouvelle fenêtre et passer des données
    private void openNewWindow(String message) {
        Stage newWindow = new Stage();
        Label label = new Label(message);       // Afficher le message reçu

        StackPane root = new StackPane(label);
        Scene scene = new Scene(root, 200, 100);
        newWindow.setScene(scene);

        newWindow.setTitle("New Window with Data");
        newWindow.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

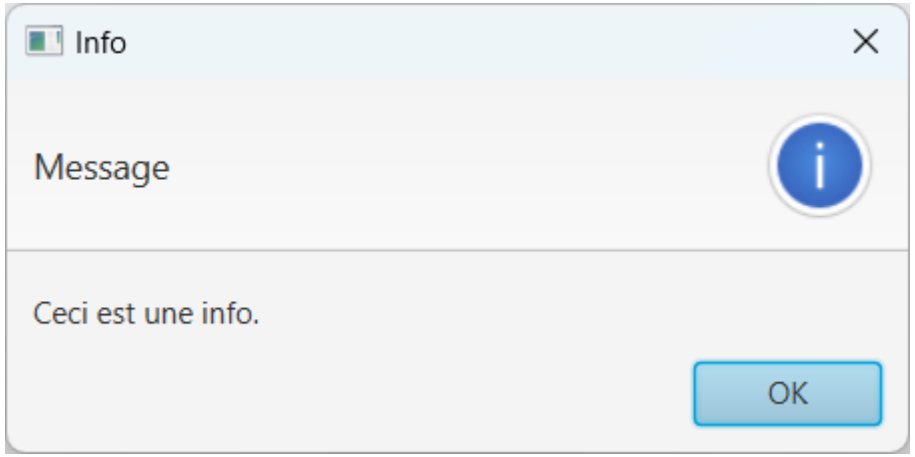
Boîtes de dialogues prédéfinies

En JavaFX, il existe plusieurs boîtes de dialogue prédéfinies prêtes à l’emploi. Elles sont pratiques pour afficher des messages, demander une confirmation ou saisir des données.

Alert – message d’information

```
import javafx.scene.control.Alert;
import javafx.scene.control.Alert.AlertType;

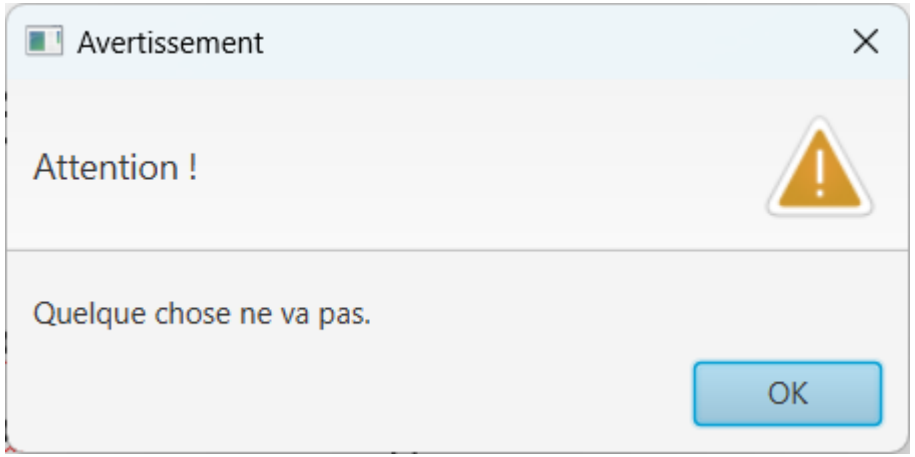
Alert alert = new Alert(AlertType.INFORMATION);
alert.setTitle("Info");
alert.setHeaderText("Message");
alert.setContentText("Ceci est une info.");
alert.showAndWait();
```



Alert – message d’information

```
import javafx.scene.control.Alert;
import javafx.scene.control.Alert.AlertType;

Alert alert = new Alert(AlertType.WARNING);
alert.setTitle("Avertissement");
alert.setHeaderText("Attention !");
alert.setContentText("Quelque chose ne va pas.");
alert.showAndWait();
```



Alert – message d’erreur

```
import javafx.scene.control.Alert;
import javafx.scene.control.Alert.AlertType;

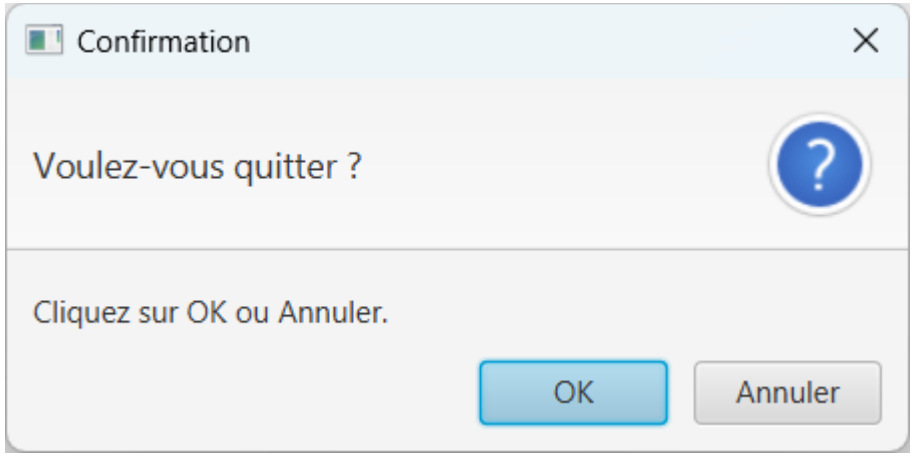
Alert alert = new Alert(AlertType.ERROR);
alert.setTitle("Erreur");
alert.setHeaderText("Erreur critique");
alert.setContentText("Impossible de continuer.");
alert.showAndWait();
```



Alert – message de confirmation

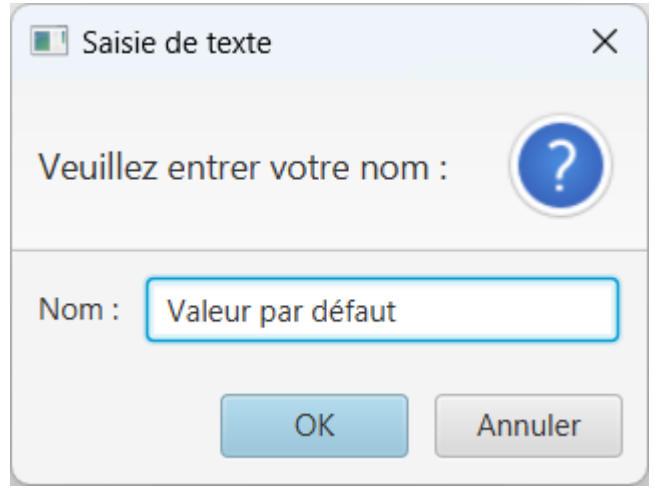
```
Alert alert = new Alert(AlertType.CONFIRMATION);
alert.setTitle("Confirmation");
alert.setHeaderText("Voulez-vous quitter ?");
alert.setContentText("Cliquez sur OK ou Annuler.");

// Affiche la boîte et récupère la réponse
Optional<ButtonType> result = alert.showAndWait();
if (result.isPresent() && result.get() == ButtonType.OK) {
    // Si l'utilisateur a cliqué sur OK → on ferme la fenêtre
    stage.close();
} else {
    // Sinon → on affiche un message dans la console
    System.out.println("L'utilisateur a annulé.");
}
```



TextInputDialog

Sert à demander une entrée texte à l'utilisateur.




```
import javafx.scene.control.TextInputDialog;

TextInputDialog dialog = new TextInputDialog("Valeur par défaut");
dialog.setTitle("Saisie de texte");
dialog.setHeaderText("Veuillez entrer votre nom :");
dialog.setContentText("Nom :");
Optional<String> result = dialog.showAndWait();
result.ifPresent(name -> System.out.println("Nom saisi : " + name));
```

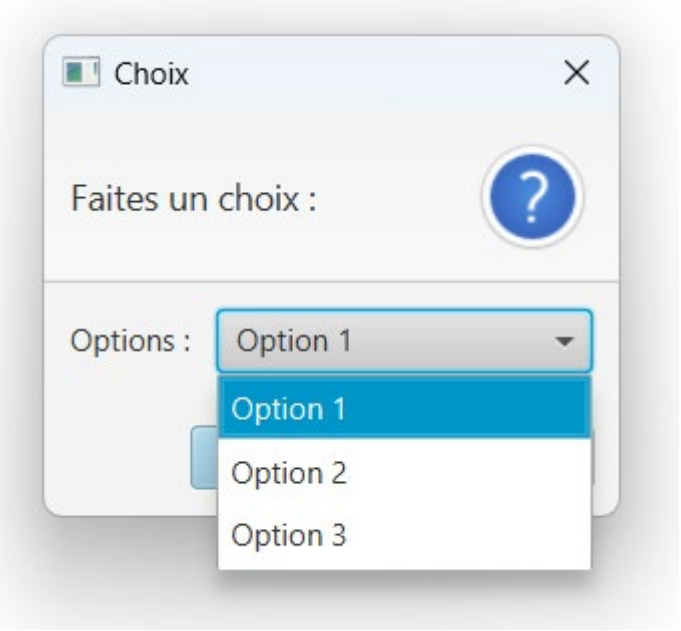
Optional<String> : type de retour de showAndWait()
L'utilisateur a fait un choix, mais il peut aussi avoir cliqué sur Annuler ou fermé la boîte de dialogue. → JavaFX renvoie un Optional<T> pour obliger à gérer ces deux cas (valeur présente ou absente).

ChoiceDialog

Permet de proposer une liste de choix prédéfinis à l'utilisateur.

```
import javafx.scene.control.ChoiceDialog;


List<String> options = Arrays.asList("Option 1", "Option 2", "Option 3");
ChoiceDialog<String> dialog = new ChoiceDialog<>("Option 1", options);
dialog.setTitle("Choix");
dialog.setHeaderText("Faites un choix :");
dialog.setContentText("Options :");
Optional<String> result = dialog.showAndWait();
result.ifPresent(choice -> System.out.println("Choix : " + choice));
```



Sélecteurs de fichiers et de dossiers

JavaFX fournit des boîtes de dialogue pour sélectionner des fichiers ou des dossiers à travers les classes **FileChooser** et **DirectoryChooser**. Ces boîtes de dialogue permettent à l'utilisateur de naviguer dans le système de fichiers pour ouvrir ou sauvegarder des fichiers.

Sélecteurs de fichiers

FileChooser est utilisé pour sélectionner un fichier à ouvrir. Il affiche une boîte de dialogue où l'utilisateur peut choisir un fichier sur son système.

```
import javafx.application.Application;
import javafx.stage.FileChooser;
import javafx.stage.Stage;
import java.io.File;

public class FileChooserExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Open Resource File");

        // Configuration du filtre d'extension (par exemple, pour
        // sélectionner uniquement les fichiers texte)
        fileChooser.getExtensionFilters().add(new FileChooser.ExtensionFilter("Text Files", "*.txt"));

        // Afficher la boîte de dialogue
        File selectedFile = fileChooser.showOpenDialog(primaryStage);

        if (selectedFile != null) {
            System.out.println("Fichier sélectionné : " + selectedFile.getPath());
        }
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Sélecteurs de dossiers

DirectoryChooser permet de sélectionner un dossier entier plutôt qu'un fichier spécifique. Il est utile lorsque vous devez spécifier un emplacement de sauvegarde ou charger des fichiers d’un répertoire spécifique.

```
import javafx.application.Application;
import javafx.stage.DirectoryChooser;
import javafx.stage.Stage;
import java.io.File;

public class DirectoryChooserExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        DirectoryChooser directoryChooser = new DirectoryChooser();
        directoryChooser.setTitle("Select Folder");
        File selectedDirectory = directoryChooser.showDialog(primaryStage);

        if (selectedDirectory != null) {
            System.out.println("Dossier sélectionné : " + selectedDirectory.getPath());
        }
    }

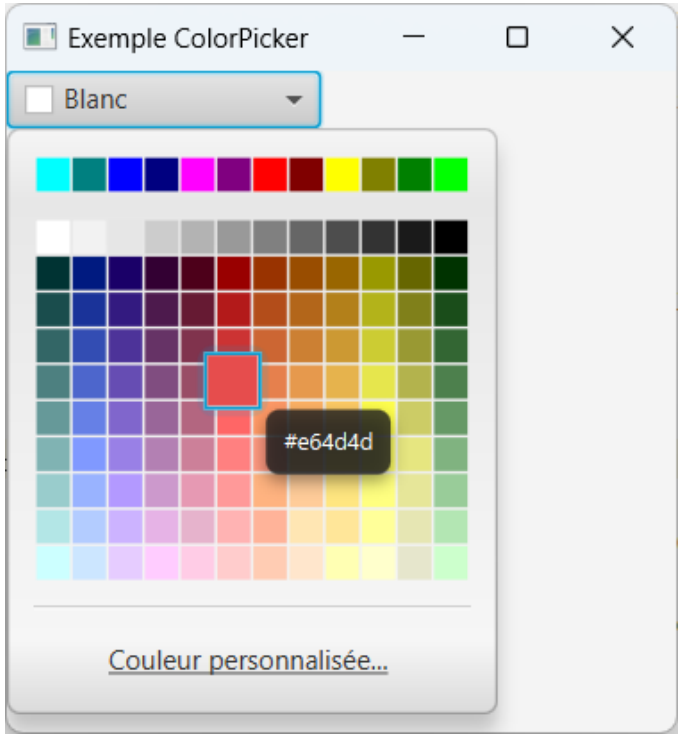
    public static void main(String[] args) {
        launch(args);
    }
}
```

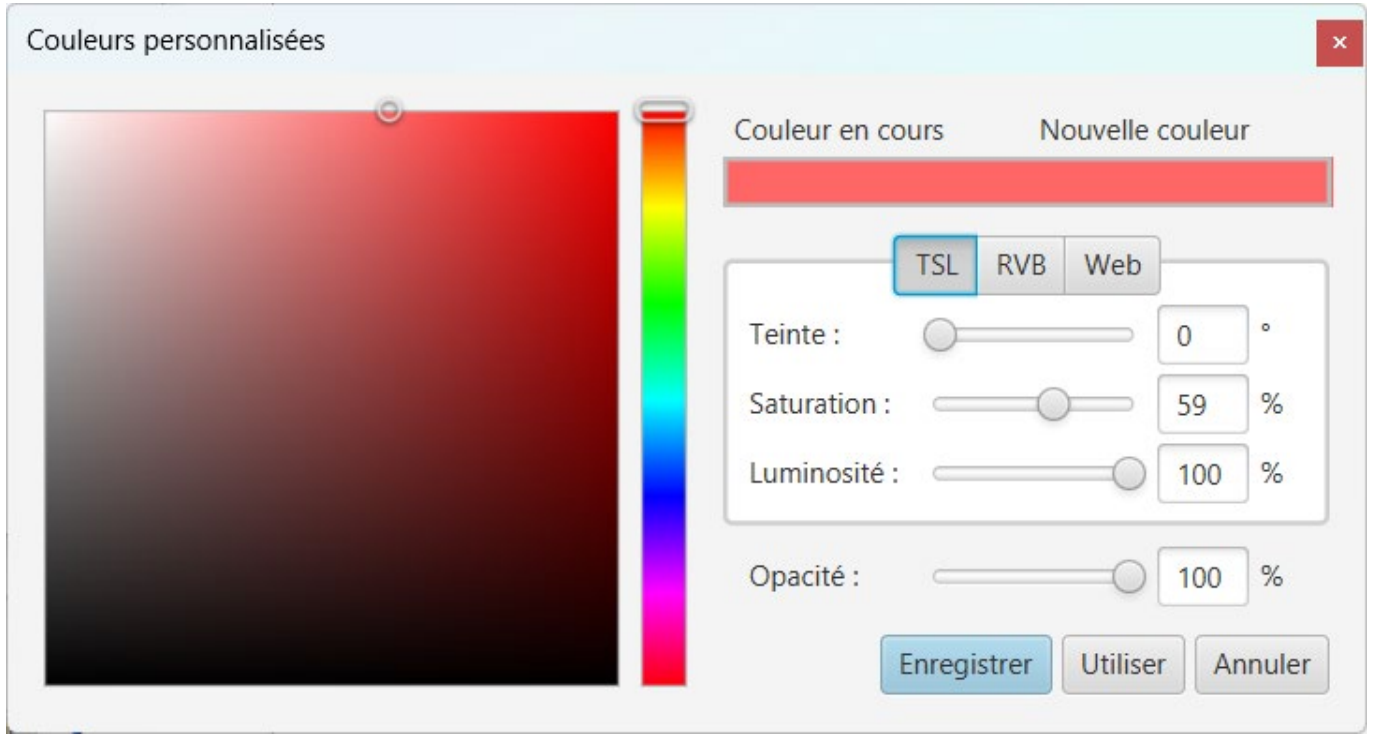
Sélecteur de couleur

https://docs.oracle.com/javafx/2/ui_controls/color-picker.htm

```
import javafx.scene.control.ColorPicker;

ColorPicker colorPicker = new ColorPicker();
Color value = colorPicker.getValue();
```





Scene Builder

Application visuelle développée à l'origine par Oracle, aujourd'hui maintenue par Gluon.

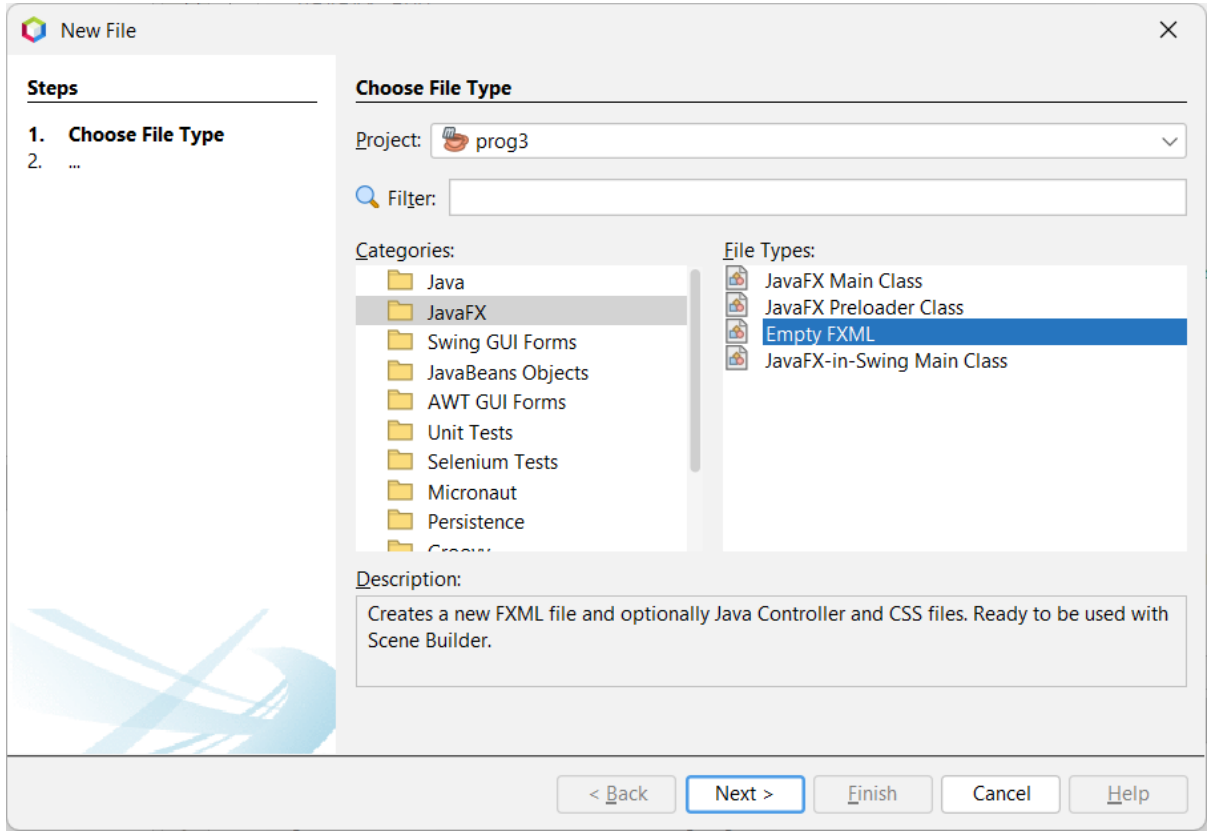
Elle permet de créer des interfaces JavaFX de manière graphique par glisser-déposer (boutons, labels, champs de texte, layouts…).

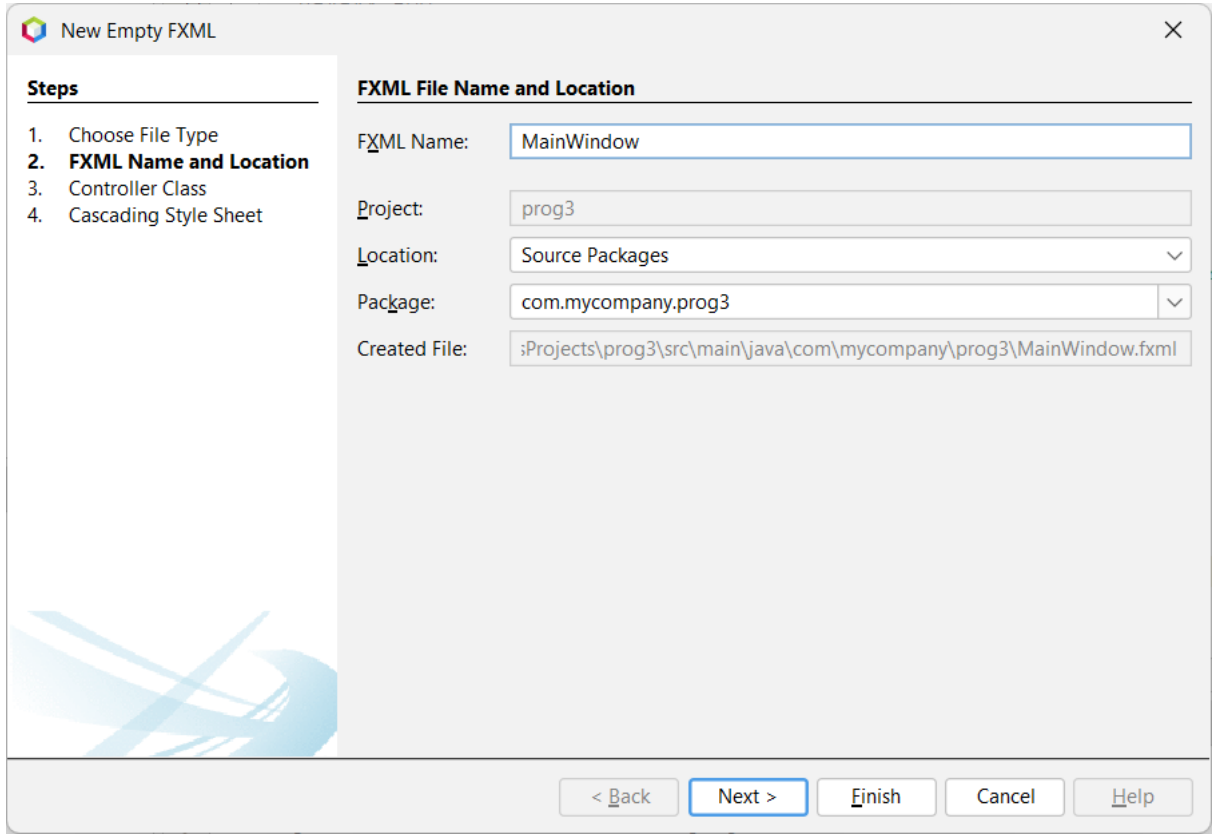
Le résultat est sauvegardé dans un fichier FXML décrivant l'interface au format XML.

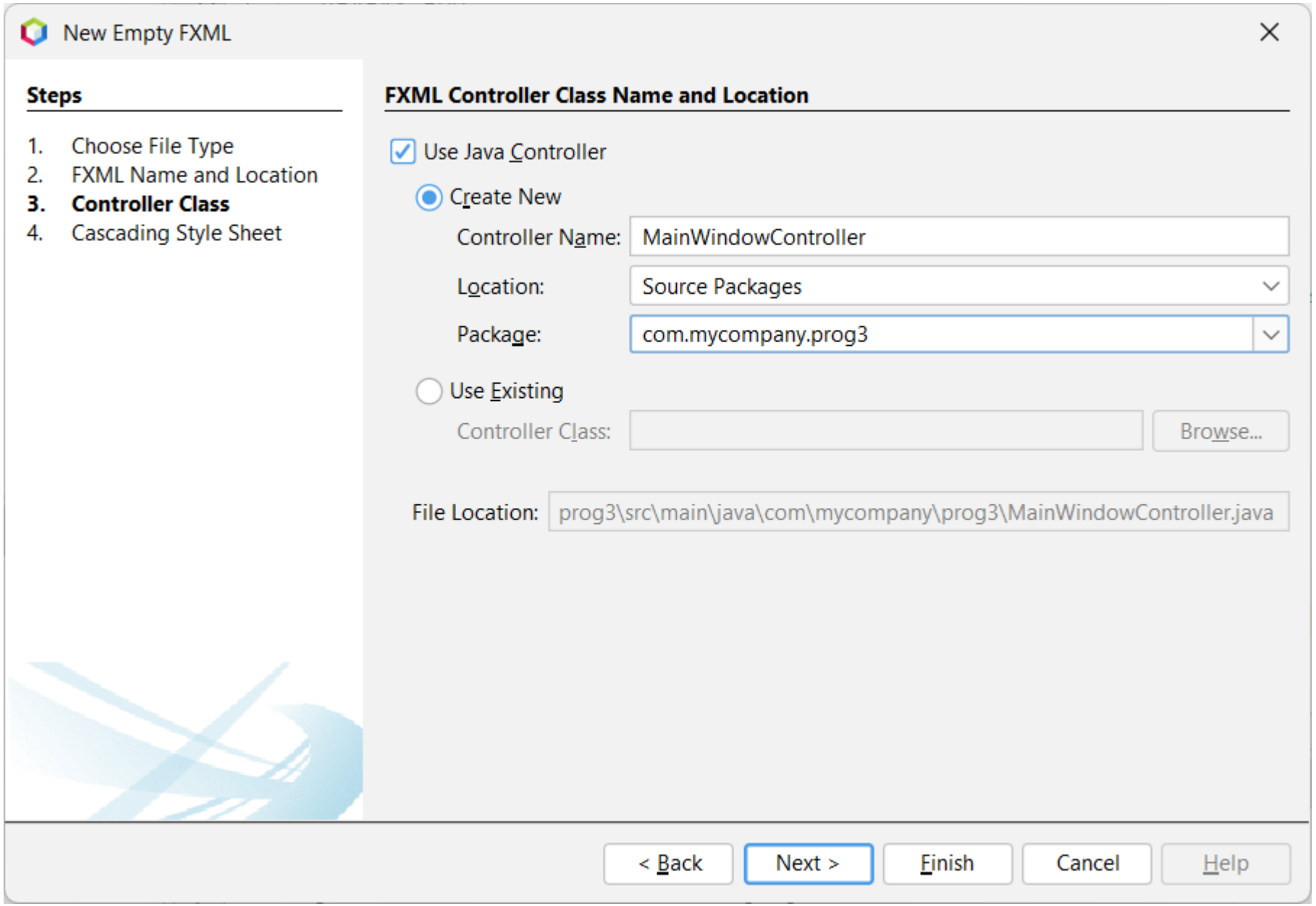
On peut ensuite lier ce FXML à un contrôleur Java pour gérer les événements et la logique métier.

→ Séparation claire entre l'interface (FXML) et la logique (Java) → bonne pratique MVC.

Scene Builder






New Empty FXML
Steps
1. Choose File Type
2. FXML Name and Location
3. Controller Class
4. Cascading Style Sheet
FXML Controller Class Name and Location
Use Java Controller
Create New
Controller Name: MainWindowController
Location: Source Packages
Package: com.mycompany.prog3
Use Existing
Controller Class:
Browse...
File Location: prog3\src\main\java\com\mycompany\prog3\MainWindowController.java
< Back
Next >
Finish
Cancel
Help

sample.fxml (Vue / créée avec Scene Builder)

```
<Button fx:id="btnClick" onAction="#handleButtonClick" />
<Label fx:id="labelMessage" text="Bonjour !" />

fx:controller="com.example.MonControleur"
```

Le FXML déclare l'UI (composants, fx:id, handlers) et référence le contrôleur.



MonControleur.java (Contrôleur)

```
@FXML private Button btnClick;
@FXML private Label labelMessage;

@FXMLprivate void handleButtonClick() {
        labelMessage.setText("Bouton cliqué !");
     }
```

Les champs @FXML sont injectés via les fx:id; les méthodes @FXML répondent aux événements déclarés dans le FXML.



MainApp.java (Application)

```
FXMLLoader loader = new FXMLLoader(getClass().getResource("sample.fxml"));
Parent root = loader.load();
primaryStage.setScene(new Scene(root)); primaryStage.show();
```

L'application charge le FXML, crée la Scene et affiche le Stage.

sample.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<AnchorPane xmlns:fx="http://javafx.com/fxml" fx:controller="com.example.MonControleur">
  <children>
    <Button fx:id="btnClick" layoutX="100" layoutY="80" text="Cliquez-moi"
            onAction="#handleButtonClick"/>
    <Label fx:id="labelMessage" layoutX="100" layoutY="40" text="Bonjour !" />
  </children>
</AnchorPane>
```

L'attribut **fx:controller="com.example.MonControleur"** fait le lien avec la classe Java **MonControleur**.

fx:id="btnClick" permet de retrouver ce bouton dans le contrôleur.

onAction="#handleButtonClick" appelle la méthode **handleButtonClick()** du contrôleur.

MonControleur.java

```
package com.example;

import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.Label;

public class MonControleur {

    @FXML
    private Button btnClick;            // lié via fx:id="btnClick"

    @FXML
    private Label labelMessage;         // lié via fx:id="labelMessage"

    @FXML
    private void handleButtonClick() {
        labelMessage.setText("Bouton cliqué !");
    }
}
```

Les annotations **@FXML** permettent à JavaFX d'injecter les composants du FXML dans le contrôleur.

MainApp.java

```
package com.example;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.scene.layout.AnchorPane;
import javafx.stage.Stage;

public class MainApp extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception {
        // Charger le fichier FXML
        FXMLLoader loader = new FXMLLoader(getClass().getResource("sample.fxml"));
        AnchorPane root = loader.load();

        // Créer la scène
        Scene scene = new Scene(root, 400, 200);
        primaryStage.setTitle("Exemple JavaFX + FXML");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

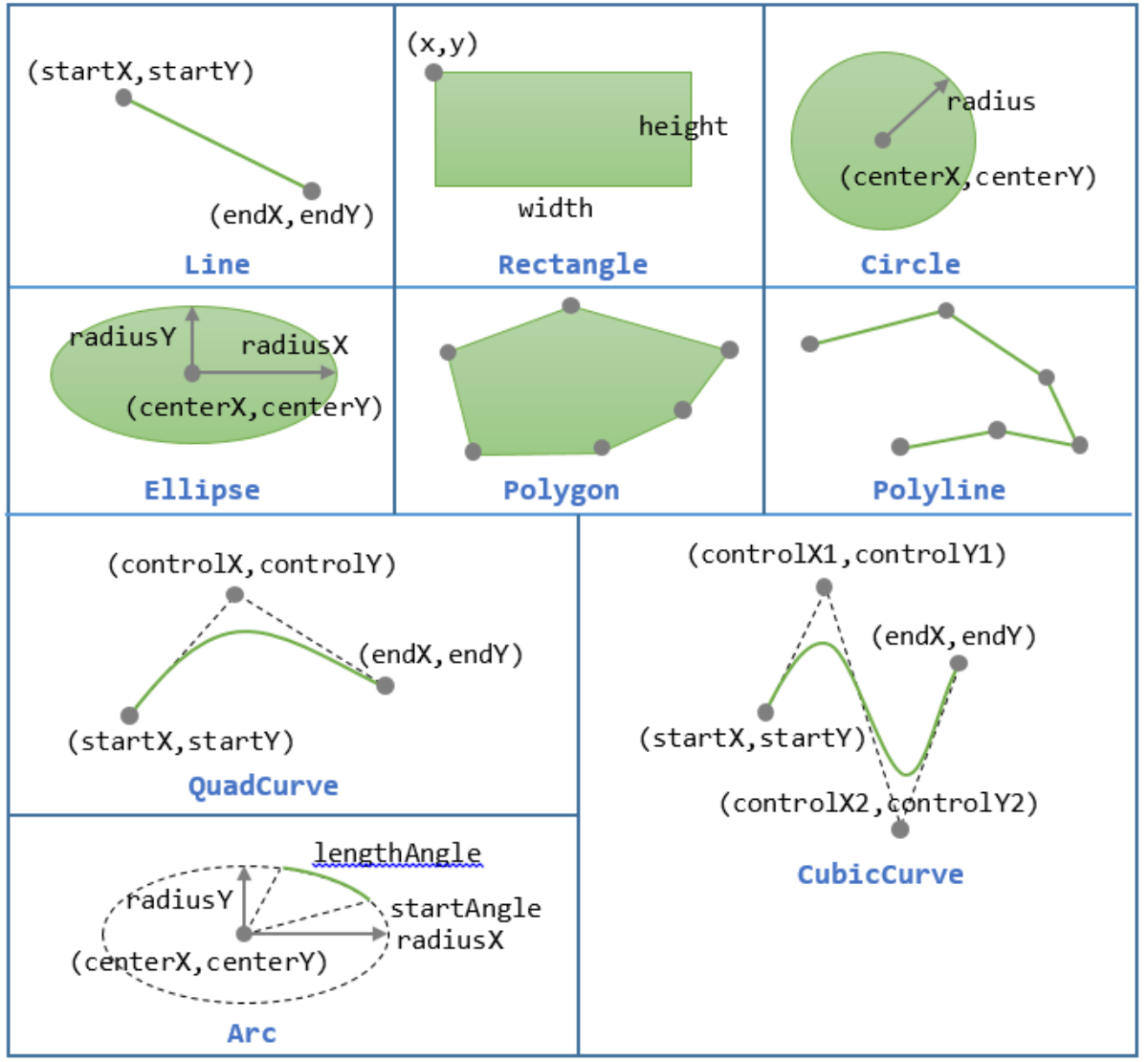
    public static void main(String[] args) {
        launch(args);
    }
}
```

FXMLLoader lit le fichier FXML, instancie les objets graphiques, et relie automatiquement le contrôleur (fx:controller) avec le code Java.

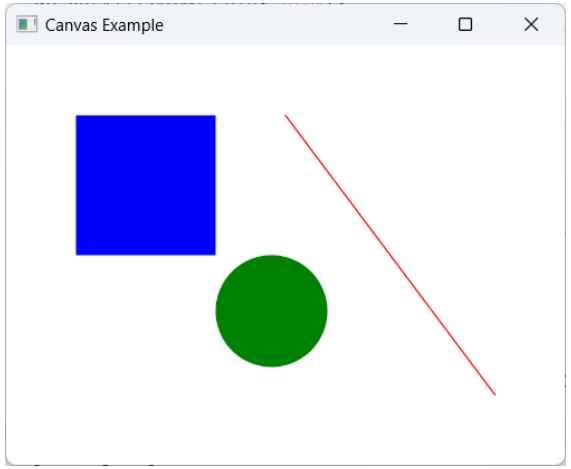
Graphismes

JavaFX permet d'afficher des graphismes avec la classe **Canvas**, qui fournit une surface pour dessiner des éléments graphiques.

Pour dessiner sur un Canvas, on utilise un objet **GraphicsContext**, qui donne accès à diverses méthodes de dessin telles que des lignes, des formes géométriques, du texte, et des images.



Exemple :




```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.layout.StackPane;
import javafx.scene.paint.Color;
import javafx.stage.Stage;

public class CanvasExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Création d'un Canvas de taille 400x300
        Canvas canvas = new Canvas(400, 300);
        // Récupération du GraphicsContext
        GraphicsContext gc = canvas.getGraphicsContext2D();

        // Dessin d'un rectangle
        gc.setFill(Color.BLUE);
        gc.fillRect(50, 50, 100, 100); // (x, y, largeur, hauteur)

        // Dessin d'une ligne
        gc.setStroke(Color.RED);
        gc.strokeLine(200, 50, 350, 250); // (x1, y1, x2, y2)

        // Dessin d'un cercle
        gc.setFill(Color.GREEN);
        gc.fillOval(150, 150, 80, 80); // (x, y, largeur, hauteur)

        StackPane root = new StackPane();
        root.getChildren().add(canvas);

        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("Canvas Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Propriétés des formes

Vous pouvez personnaliser l’apparence des formes dessinées en utilisant des méthodes comme :

setStroke(Color color) pour définir la couleur des lignes.

setFill(Color color) pour définir la couleur de remplissage des formes.

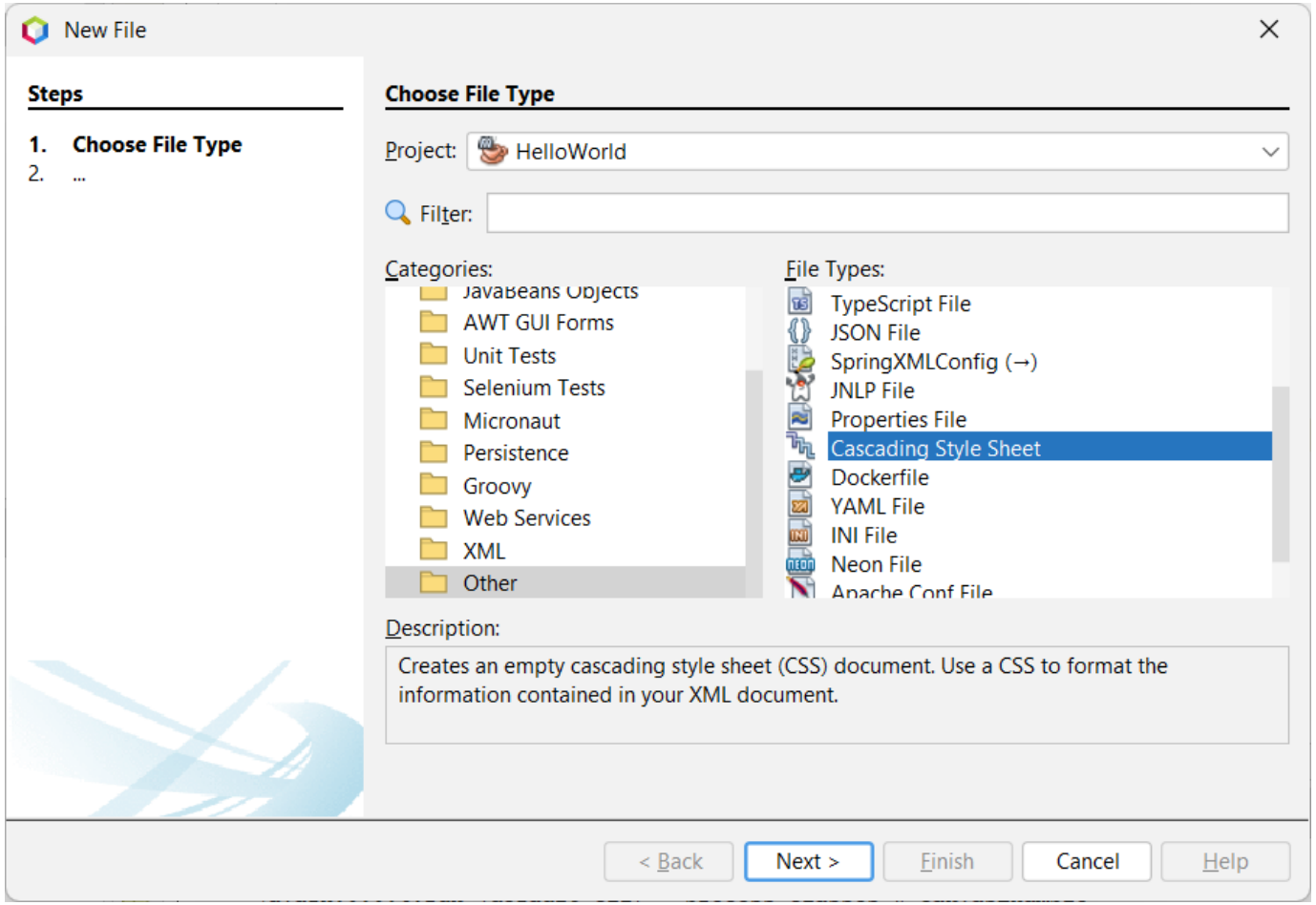
setLineWidth(double width) pour ajuster l'épaisseur des lignes.

Styles et personnalisation

JavaFX permet de styliser l'interface utilisateur à l'aide de feuilles de style CSS (Cascading Style Sheets). Tout comme pour le développement web, le CSS dans JavaFX permet de personnaliser l'apparence des éléments de l'interface, comme les boutons, les labels, les zones de texte, etc.

Pour ajouter un fichier CSS à votre projet :

Dans Netbeans, faire un clic droit sur le nom du projet → nouveau → catégorie « Other » → Cascading Style Sheet.



Association de CSS à une application JavaFX

Les feuilles de style CSS peuvent être appliquées à une application JavaFX en les associant à une Scene ou à des nœuds spécifiques.

Exemple de lien entre un fichier CSS et une scène :

```
Scene scene = new Scene(root, 300, 250);
scene.getStylesheets().add("styles.css");
```

Cela signifie que tous les nœuds de la Scene utiliseront les règles définies dans le fichier styles.css

Sélecteurs CSS dans JavaFX

Les sélecteurs CSS permettent de cibler des composants spécifiques dans l'application.

Sélecteur de type : Ce sélecteur cible tous les éléments d’un type donné.

Par exemple, tous les boutons :

```
Button {
    -fx-background-color: #3498db;
    -fx-text-fill: white;
    -fx-font-size: 14px;
}
```

Code CSS

Sélecteurs CSS dans JavaFX

Sélecteur d'ID : Si vous donnez un ID unique à un composant, vous pouvez le cibler avec un sélecteur CSS d'ID :

```
button.setId("myButton");
```

Code Java

```
#myButton {
    -fx-background-color: #e74c3c;
}
```

Code CSS

Sélecteurs CSS dans JavaFX

Sélecteur de classe : Vous pouvez également définir une classe CSS et l'appliquer à plusieurs composants :

```
button.getStyleClass().add("blue-button");
```

Code Java

```
.blue-button {
    -fx-background-color: #3498db;
    -fx-text-fill: white;
}
```

Code CSS

Propriétés CSS spécifiques à JavaFX

JavaFX utilise des propriétés spécifiques pour les styles, qui commencent souvent par le préfixe **-fx-**

Quelques exemples de propriétés courantes :

-fx-background-color : Définit la couleur de fond.

-fx-text-fill : Définit la couleur du texte.

-fx-padding : Définit le remplissage intérieur du composant.

-fx-border-color : Définit la couleur de la bordure.

-fx-font-size : Change la taille de la police.

Exemple

Code CSS

```
Button {
    -fx-background-color: #2ecc71;
    -fx-text-fill: white;
    -fx-font-size: 16px;
    -fx-border-radius: 5px;
    -fx-background-radius: 5px;
}

TextField {
    -fx-border-color: #3498db;
    -fx-border-width: 2px;
}
```

```
/* Style pour un TextField avec la classe 'important' */
.important {
    -fx-border-color: #e74c3c;
    -fx-border-width: 3px;
}


/* Style pour un label spécifique avec un ID */
#mainLabel {
    -fx-text-fill: #2ecc71;
    -fx-font-weight: bold;
}
```